Deep Kalman Filters

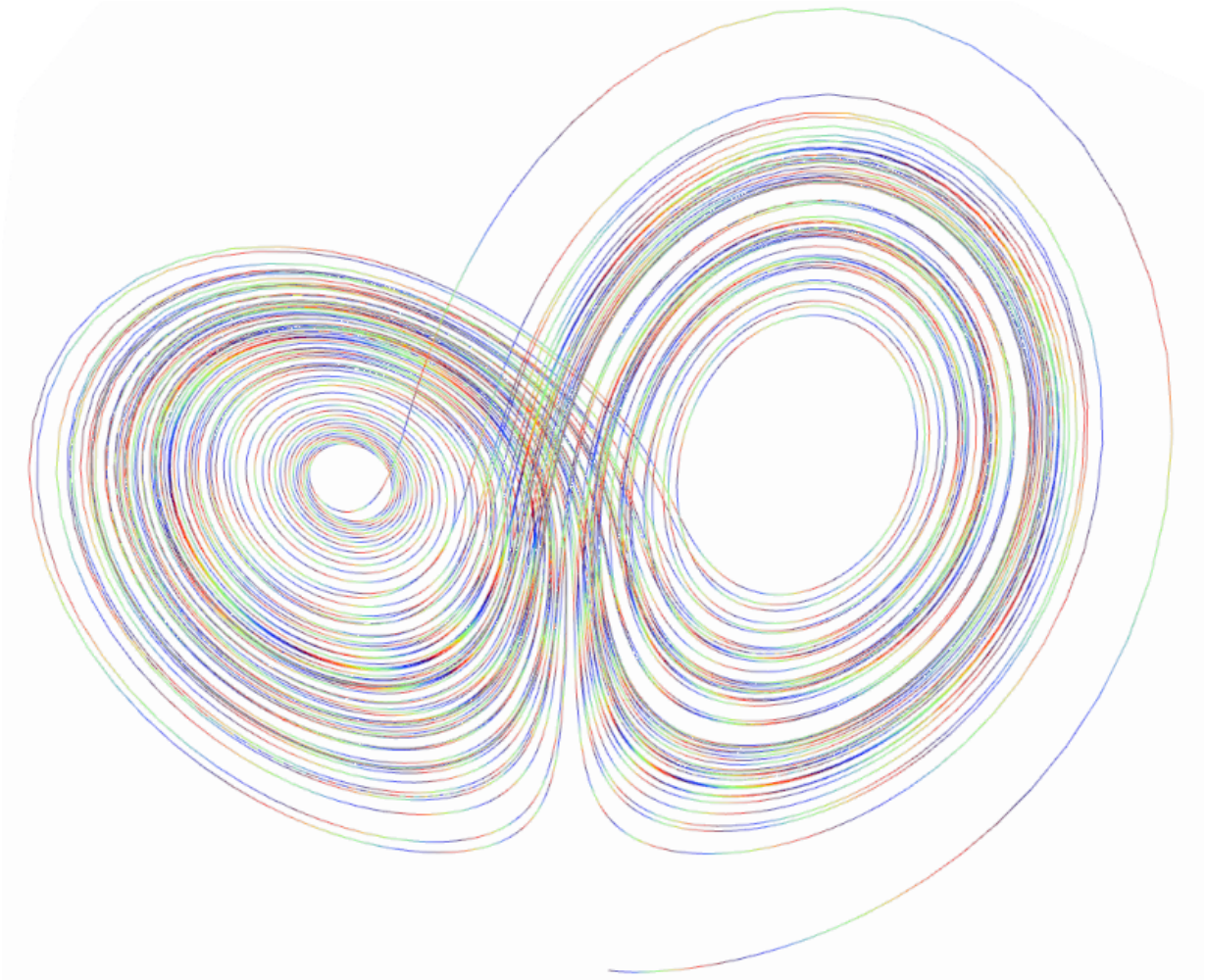
Christophe Louargant

Conservatoire **N**ational des **A**rts et **M**étiers

Paris

RCP 217

December 2024



License

This work is licensed under the Creative Commons Attribution 4.0 International License.

CONTENTS

1 Introduction **5**

2 Kalman Filter **6**
2.1 Introduction . . . 6
2.2 System Model . . . 6
2.3 Kalman Filter Algorithm . . . 6
2.3.1 Prediction Step . . . 6
2.3.2 Update Step . . . 7
2.4 The probabilistic reading of the Kalman filter . . . 7
2.5 Summary . . . 7

3 Overview of VAEs **8**
3.1 Introduction . . . 8
3.2 The model . . . 8
3.3 The Evidence Lower Bound . . . 9
3.3.1 Why it is a lower bound . . . 9
3.3.2 Reconstruction and regularisation terms . . . 9
3.4 Training the VAE . . . 9
3.4.1 Reparameterisation trick . . . 9
3.5 Summary . . . 9

4 Deep Kalman Filters **11**
4.1 Introduction . . . 11
4.2 Model Specification . . . 11
4.2.1 Generative model . . . 12
4.3 The posterior and the inference model . . . 12
4.3.1 The four inference models of the paper . . . 12
4.3.2 Structure of the true posterior . . . 12
4.4 Evidence Lower Bound . . . 13
4.5 DKF Inference Model Design . . . 13
4.5.1 Reparameterisation trick . . . 14
4.5.2 Overall design . . . 14
4.6 DKF Generative Model Design . . . 15
4.7 Estimating the ELBO . . . 15
4.7.1 The difficulty . . . 15
4.7.2 Ancestral sampling . . . 15
4.7.3 The estimator . . . 16
4.7.4 Why compute the KL terms in closed form? . . . 16
4.7.5 Differentiability . . . 16
4.8 Training the DKF . . . 16
4.8.1 The loss function . . . 17

4.9 Implementation . . . 18
4.9.1 The forward pass . . . 18
4.9.2 The `predict` function . . . 18
4.10 The Autoregressive DKF (AR DKF) . . . 19
4.10.1 Motivation . . . 19
4.10.2 Generative model . . . 19
4.10.3 What the new edges change for inference . . . 19
4.10.4 Prediction . . . 20
4.11 The FAR DKF and BAR DKF models . . . 20
4.11.1 Forward Autoregressive DKF (FAR DKF) . . . 20
4.11.2 Bidirectional Autoregressive DKF (BAR DKF) . . . 21
4.11.3 Summary . . . 22

5 Conclusion **23**

Appendices **24**
A Understanding the Backward LSTM . . . 25
B Detailed Derivation of the ELBO . . . 26
C Detailed Mathematical Development of the Reconstruction Term . . . 28
D Detailed Mathematical Development of the Regularization Term . . . 30
E Combiner in the Deep Kalman Filter (DKF) . . . 32
E.1 Linear Combiner . . . 32
E.2 MLP Combiner . . . 33
E.3 Combiner Factory . . . 34
E.4 Summary of the Combiner's Role in the DKF . . . 35
F Results of Deep Kalman Filter Models . . . 36

Annex **49**
I Bayesian Networks: Paths and D-Separation . . . 49

LIST OF FIGURES

3.1 Variational Auto-Encoder 8

4.1 DKF model as a Bayesian network. The inputs $\mathbf{u}_t$ are always observed. 11
4.2 Backward RNN: the output at position t summarises $\mathbf{x}_{t:T}$. 14
4.3 DKF inference model 14
4.4 DKF generative model 15
4.5 DKF model: inference network (left and centre) and generative networks (right). 17
4.6 DKF implementation classes 18
4.7 DKF model versus AR DKF model (blue: the added edges). 19
4.8 AR DKF model 21
4.9 FAR DKF: the forward RNN `q_f_rnn` feeds the combiner of the inference network. 21

1 Backward LSTM. In this figure the hidden states are indexed by their position in the *flipped* sequence: h_k is $\mathbf{b}_{T-k+1}$ in the notation of this appendix. 25
2 DKF reconstruction and prediction on two sine-wave datasets. 36
3 Examples of AR DKF Reconstruction and Prediction on two simple sine wave datasets. . . . 37
4 Reconstruction and prediction MSE of the DKF and the AR DKF on the two sine-wave datasets. 37
5 Lorenz attractor - Range [4500, 6500] 39
6 Examples of Reconstruction and Prediction for the AR DKF model. 39
7 Examples of Reconstruction and Prediction for the FAR DKF model. 40
8 Examples of Reconstruction and Prediction for the BAR DKF model. 40
9 Reconstruction and Prediction MSE Comparison for AR DKF, FAR DKF, and BAR DKF on dataset 1. 40
10 MSE Comparison for AR DKF, FAR DKF, and BAR DKF for each prediction points of test sequences of dataset 1. 41
11 Lorenz attractor - Range [8600, 10500] 41
12 Examples of Reconstruction and Prediction for the AR DKF model. 42
13 Examples of Reconstruction and Prediction for the FAR DKF model. 42
14 Examples of Reconstruction and Prediction for the BAR DKF model. 42
15 Reconstruction and Prediction MSE Comparison for AR DKF, FAR DKF, and BAR DKF on dataset 2. 43
16 MSE Comparison for AR DKF, FAR DKF, and BAR DKF for each prediction points of test sequences of dataset 2. 43
17 Lorenz attractor - Range [11000, 13000] 44
18 Examples of Reconstruction and Prediction for the AR DKF model. 44
19 Examples of Reconstruction and Prediction for the FAR DKF model. 44
20 Examples of Reconstruction and Prediction for the BAR DKF model. 45
21 Reconstruction and Prediction MSE Comparison for AR DKF, FAR DKF, and BAR DKF on dataset 3. 45

22 MSE Comparison for AR DKF, FAR DKF, and BAR DKF for each prediction points of test sequences of dataset 3. 46

Chapter 1

Introduction

The Deep Kalman Filter (DKF), introduced by Krishnan, Shalit and Sontag in 2015 [1], is a sequential latent-variable model. It keeps the structure of the Kalman filter, a latent state that evolves as a Markov chain and emits one observation per time step, but replaces the linear-Gaussian transition and emission models by Gaussian distributions whose parameters are computed by neural networks. Since exact posterior inference is then intractable, the model is trained like a Variational Auto-Encoder (VAE): an inference network approximates the posterior over the latent trajectory, and the parameters of both networks are learned jointly by maximising the Evidence Lower Bound (ELBO).

This report presents the mathematics of the DKF, my implementation of it[1], and experiments on noisy sine waves and on sequences generated by the Lorenz attractor[2]. It is organised as follows.

- Chapter 2 recalls the Kalman filter. It is the linear-Gaussian special case in which the posterior that the DKF approximates can be computed exactly.
- Chapter 3 recalls the VAE and, in particular, *why* the ELBO is a lower bound on the log-likelihood.
- Chapter 4 is the core of the report: the DKF model, the structure of its true posterior (derived with d-separation), the factorised ELBO, how that ELBO is estimated by sampling, the implementation, and three extensions I designed: the autoregressive DKF (AR DKF) and its two variants FAR DKF and BAR DKF.
- The appendices give the complete derivations and the experimental results. The annex recalls d-separation.

[1] The code is available on GitHub.
[2] The cover page shows a realisation of the Lorenz attractor used to test the models.

Chapter 2

Kalman Filter

2.1 Introduction

The Kalman filter estimates the state of a linear dynamical system from a sequence of noisy measurements. In the robotics and signal-processing literature the state at time t is usually written $\mathbf{x}_t$ and the measurement $\mathbf{z}_t$ (see, e.g., [2]). The Deep Kalman Filter paper [1] follows the VAE convention instead, and I do the same throughout this report: $\mathbf{z}_t$ is the *latent* state and $\mathbf{x}_t$ the *observed* measurement.

Control input $\mathbf{u}_{t-1}$ versus $\mathbf{u}_t$

Krishnan et al. write the control input that drives the transition from $t-1$ to t as $\mathbf{u}_{t-1}$, but when they define their variational models, $\mathbf{u}_t$ is synchronous with $\mathbf{z}_t$. I keep the convention of the Kalman-filter literature and write $\mathbf{u}_t$ for the input acting on the transition into $\mathbf{z}_t$. This is only a matter of indexing.

2.2 System Model

The system is described by an initial belief, a state-transition equation and an observation equation.

Initial state. $\mathbf{z}_0 \sim \mathcal{N}(\boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0)$.

State transition. For $t \geq 1$,

$$\mathbf{z}_t = \mathbf{G}_t \mathbf{z}_{t-1} + \mathbf{B}_t \mathbf{u}_t + \boldsymbol{\varepsilon}_t, \qquad \boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_t),$$

where $\mathbf{G}_t$ is the state-transition matrix, $\mathbf{B}_t$ the control-input matrix, $\mathbf{u}_t$ the (known) control input and $\boldsymbol{\varepsilon}_t$ the process noise. Equivalently,

$$p(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{u}_t) = \mathcal{N}(\mathbf{z}_t;\ \mathbf{G}_t \mathbf{z}_{t-1} + \mathbf{B}_t \mathbf{u}_t,\ \boldsymbol{\Sigma}_t). \tag{2.1}$$

Observation. For $t \geq 1$,

$$\mathbf{x}_t = \mathbf{F}_t \mathbf{z}_t + \boldsymbol{\eta}_t, \qquad \boldsymbol{\eta}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Gamma}_t),$$

where $\mathbf{F}_t$ is the observation matrix and $\boldsymbol{\eta}_t$ the measurement noise. Equivalently,

$$p(\mathbf{x}_t \mid \mathbf{z}_t) = \mathcal{N}(\mathbf{x}_t;\ \mathbf{F}_t \mathbf{z}_t,\ \boldsymbol{\Gamma}_t). \tag{2.2}$$

2.3 Kalman Filter Algorithm

Write $\hat{\mathbf{z}}_t$ and $\mathbf{P}_t$ for the mean and covariance of the state given the measurements up to time t. The filter alternates two steps.

2.3.1 Prediction Step

- **Predicted (a priori) mean:** $\bar{\mathbf{z}}_t = \mathbf{G}_t \hat{\mathbf{z}}_{t-1} + \mathbf{B}_t \mathbf{u}_t$.
- **Predicted (a priori) covariance:** $\bar{\mathbf{P}}_t = \mathbf{G}_t \mathbf{P}_{t-1} \mathbf{G}_t^\top + \boldsymbol{\Sigma}_t$.

These are the mean and covariance of the state at time t given the measurements up to time $t-1$.

2.3.2 Update Step

- **Innovation (measurement residual):** $\mathbf{y}_t = \mathbf{x}_t - \mathbf{F}_t \bar{\mathbf{z}}_t$.
- **Innovation covariance:** $\mathbf{S}_t = \mathbf{F}_t \bar{\mathbf{P}}_t \mathbf{F}_t^\top + \mathbf{\Gamma}_t$.
- **Kalman gain:** $\mathbf{K}_t = \bar{\mathbf{P}}_t \mathbf{F}_t^\top \mathbf{S}_t^{-1}$.
- **Updated (a posteriori) mean:** $\hat{\mathbf{z}}_t = \bar{\mathbf{z}}_t + \mathbf{K}_t \mathbf{y}_t$.
- **Updated (a posteriori) covariance:** $\mathbf{P}_t = (\mathbf{I} - \mathbf{K}_t \mathbf{F}_t) \bar{\mathbf{P}}_t$.

2.4 The probabilistic reading of the Kalman filter

Because every distribution in the model is Gaussian and every map is linear, all posterior distributions are Gaussian and can be computed in closed form. The two steps above compute exactly

$$p(\mathbf{z}_t \mid \mathbf{x}_{1:t-1}, \mathbf{u}_{1:t}) = \mathcal{N}(\bar{\mathbf{z}}_t, \bar{\mathbf{P}}_t) \qquad \text{and} \qquad p(\mathbf{z}_t \mid \mathbf{x}_{1:t}, \mathbf{u}_{1:t}) = \mathcal{N}(\hat{\mathbf{z}}_t, \mathbf{P}_t).$$

The innovations also give the exact log-likelihood of the measurements,

$$\log p(\mathbf{x}_{1:T} \mid \mathbf{u}_{1:T}) = \sum_{t=1}^{T} \log \mathcal{N}(\mathbf{y}_t;\ \mathbf{0},\ \mathbf{S}_t),$$

and a backward pass (the Rauch–Tung–Striebel smoother) gives the *smoothing* distributions $p(\mathbf{z}_t \mid \mathbf{x}_{1:T}, \mathbf{u}_{1:T})$, which use the future measurements as well.

In words

In the linear-Gaussian case, "filtering" and "smoothing" are exact Bayesian inference, done with matrix algebra. The Deep Kalman Filter keeps the same probabilistic structure but makes the transition and emission non-linear. Then none of these distributions has a closed form any more. The DKF replaces exact inference by a neural network that *approximates* the smoothing posterior, and replaces the exact log-likelihood by a lower bound on it. Chapters 3 and 4 explain how.

2.5 Summary

The Kalman filter alternates a prediction step, which propagates the belief through the dynamics, and an update step, which corrects it with the new measurement. For linear-Gaussian systems this is exact inference, and the resulting estimate is optimal in the mean-squared-error sense. The Deep Kalman Filter replaces all the linear maps by non-linear maps parameterised by neural networks, at the price of approximate inference.

Chapter 3

Overview of VAEs

3.1 Introduction

This chapter recalls the Variational Auto-Encoder (VAE) and introduces the notions used for the Deep Kalman Filter in the next chapter. For a complete treatment, see Murphy's book [3].

A VAE has two components (Figure 3.1):

- an **encoder**, which maps an input $\mathbf{x}$ to a distribution over a latent variable $\mathbf{z}$;
- a **decoder**, which maps a latent value $\mathbf{z}$ to a distribution over the data space, from which a reconstruction $\hat{\mathbf{x}}$ is obtained.

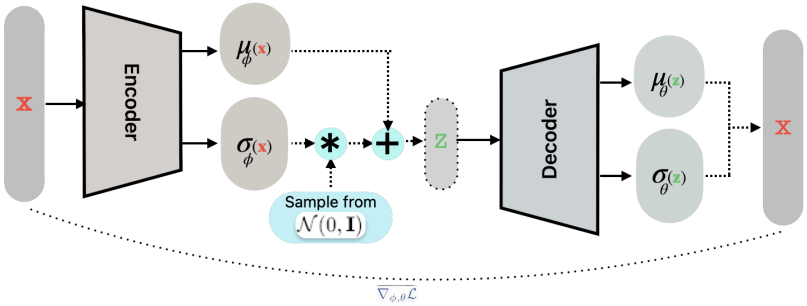


Figure 3.1: Variational Auto-Encoder

3.2 The model

Let $\{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(N)}\}$ be a dataset of independent data points. Each data point is assumed to be generated by the following process.

Generative model (decoder).

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z};\ \mathbf{0}, \mathbf{I}), \qquad p_\theta(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}\big(\mathbf{x};\ \boldsymbol{\mu}_\theta(\mathbf{z}),\ \mathrm{diag}(\boldsymbol{\sigma}^2_\theta(\mathbf{z}))\big),$$

where $\boldsymbol{\mu}_\theta$ and $\boldsymbol{\sigma}^2_\theta$ are computed by a neural network with parameters θ.

Inference model (encoder). The posterior $p_\theta(\mathbf{z} \mid \mathbf{x}) = p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z}) / p_\theta(\mathbf{x})$ is intractable, because $p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}$ has no closed form when the decoder is a neural network. It is approximated by

$$q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}\big(\mathbf{z};\ \boldsymbol{\mu}_\phi(\mathbf{x}),\ \mathrm{diag}(\boldsymbol{\sigma}^2_\phi(\mathbf{x}))\big),$$

where $\boldsymbol{\mu}_\phi$ and $\boldsymbol{\sigma}^2_\phi$ are computed by a second neural network with parameters ϕ. A single network serves every data point; this is called *amortised* inference.

3.3 The Evidence Lower Bound

3.3.1 Why it is a lower bound

For any distribution $q(\mathbf{z} \mid \mathbf{x})$ whose support contains that of the posterior,

$$\log p_\theta(\mathbf{x}) = \log \int q(\mathbf{z} \mid \mathbf{x}) \, \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q(\mathbf{z} \mid \mathbf{x})} \, d\mathbf{z} = \log \mathbb{E}_{q(\mathbf{z}|\mathbf{x})} \left[\frac{p_\theta(\mathbf{x}, \mathbf{z})}{q(\mathbf{z} \mid \mathbf{x})} \right]$$
$$\geq \mathbb{E}_{q(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q(\mathbf{z} \mid \mathbf{x})} \right] \;=:\; \mathcal{L}(\theta, \phi; \mathbf{x}),$$

by Jensen's inequality, since log is concave. This is the **Evidence Lower Bound** (ELBO). The size of the gap is known exactly. Writing $p_\theta(\mathbf{x}, \mathbf{z}) = p_\theta(\mathbf{x}) \, p_\theta(\mathbf{z} \mid \mathbf{x})$ inside the logarithm gives

$$\log p_\theta(\mathbf{x}) = \mathcal{L}(\theta, \phi; \mathbf{x}) + \mathrm{KL}\big(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p_\theta(\mathbf{z} \mid \mathbf{x})\big). \tag{3.1}$$

In words

The ELBO is below the log-likelihood by exactly the KL divergence between the encoder and the true posterior. Maximising the ELBO over ϕ therefore makes the encoder a better approximation of the posterior. Maximising it over θ pushes the log-likelihood up. The bound is tight when the encoder equals the posterior.

3.3.2 Reconstruction and regularisation terms

Writing $p_\theta(\mathbf{x}, \mathbf{z}) = p_\theta(\mathbf{x} \mid \mathbf{z}) \, p(\mathbf{z})$ instead gives the usual form

$$\mathcal{L}(\theta, \phi; \mathbf{x}) = \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \big[\log p_\theta(\mathbf{x} \mid \mathbf{z}) \big]}_{\text{reconstruction term}} \; - \; \underbrace{\mathrm{KL}\big(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z})\big)}_{\text{regularisation term}} .$$

- The **reconstruction term** rewards latent codes from which the decoder assigns high likelihood to the input. Its negative is the reconstruction loss.
- The **regularisation term** penalises encoders that move away from the prior. With Gaussian q_ϕ and prior it has a closed form (Appendix D).

3.4 Training the VAE

The VAE is trained by minimising $-\mathcal{L}$, averaged over the dataset, with stochastic gradient descent or one of its variants. The expectation in the reconstruction term is estimated with one (or a few) samples $\mathbf{z} \sim q_\phi(\mathbf{z} \mid \mathbf{x})$. Since a Monte Carlo average of samples from q_ϕ is an unbiased estimate of the expectation, the gradient of the estimate is an unbiased estimate of the gradient, provided the sample can be differentiated with respect to ϕ.

3.4.1 Reparameterisation trick

That is what the reparameterisation trick provides. Instead of sampling $\mathbf{z}$ from q_ϕ directly, write

$$\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),$$

where $\boldsymbol{\sigma}_\phi$ is the vector of standard deviations (the square root of the variances) and $\odot$ the element-wise product. The randomness is now in $\boldsymbol{\epsilon}$, which does not depend on ϕ, so $\mathbf{z}$ is a differentiable function of ϕ for fixed $\boldsymbol{\epsilon}$.

3.5 Summary

A VAE approximates the intractable posterior $p_\theta(\mathbf{z} \mid \mathbf{x})$ by an encoder $q_\phi(\mathbf{z} \mid \mathbf{x})$ and trains encoder and decoder together by maximising

$$\mathcal{L}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x} \mid \mathbf{z})] - \mathrm{KL}(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z})) = \log p_\theta(\mathbf{x}) - \mathrm{KL}(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p_\theta(\mathbf{z} \mid \mathbf{x})).$$

The Deep Kalman Filter applies the same recipe to a whole sequence, with a latent variable per time step linked by a Markov chain.

Chapter 4

Deep Kalman Filters

4.1 Introduction

The Deep Kalman Filter (DKF) keeps the structure of the Kalman filter but uses neural networks for the state transition, the observation model and the inference. The linear-Gaussian equations (2.1) and (2.2) are replaced by

$$\mathbf{z}_t \sim \mathcal{N}\big(\mathbf{G}_\alpha(\mathbf{z}_{t-1}, \mathbf{u}_t, \Delta_t),\ \mathbf{S}_\beta(\mathbf{z}_{t-1}, \mathbf{u}_t, \Delta_t)\big) \tag{4.1}$$

and

$$\mathbf{x}_t \sim \mathbf{\Pi}\big(\mathbf{F}_\kappa(\mathbf{z}_t)\big), \tag{4.2}$$

where $\mathbf{G}_\alpha$, $\mathbf{S}_\beta$ and $\mathbf{F}_\kappa$ are neural networks, Δ_t is the time elapsed since the previous observation, and $\mathbf{\Pi}$ is a distribution (Gaussian, Bernoulli, ...) whose parameters are output by $\mathbf{F}_\kappa$.

As for a VAE, Krishnan et al. [1] use variational inference to learn the model $p_\theta(\mathbf{x}_{1:T}, \mathbf{z}_{1:T} \mid \mathbf{u}_{1:T})$. Two things differ from a standard VAE: there is a sequence of inputs $\mathbf{u}_{1:T}$, and the latent variables $\mathbf{z}_{1:T} = \{\mathbf{z}_t\}_{t=1}^T$ are linked over time, so the observations $\mathbf{x}_{1:T}$ are not independent either.

Conditioning on known inputs

The inputs $\mathbf{u}_{1:T}$ are known and need not be random: conditioning on them simply means that the model is a family of distributions over $(\mathbf{x}, \mathbf{z})$ indexed by $\mathbf{u}$. In a graphical model, such inputs can be drawn as nodes that are always observed, and d-separation applies unchanged.

4.2 Model Specification

Latent dynamics.

$$\mathbf{z}_t \sim \mathcal{N}\big(\mathbf{G}_\alpha(\mathbf{z}_{t-1}, \mathbf{u}_t, \Delta_t),\ \mathbf{S}_\beta(\mathbf{z}_{t-1}, \mathbf{u}_t, \Delta_t)\big), \qquad t \geq 1,$$

with $\mathbf{S}_\beta$ a diagonal covariance matrix. For $t = 1$ the "previous state" is an initial value $\mathbf{z}_0$, which in my implementation is either fixed at $\mathbf{0}$ or a learned parameter.

Observation model.

$$\mathbf{x}_t \sim p(\mathbf{x}_t \mid \mathbf{z}_t) = \mathbf{\Pi}\big(\mathbf{F}_\kappa(\mathbf{z}_t)\big).$$

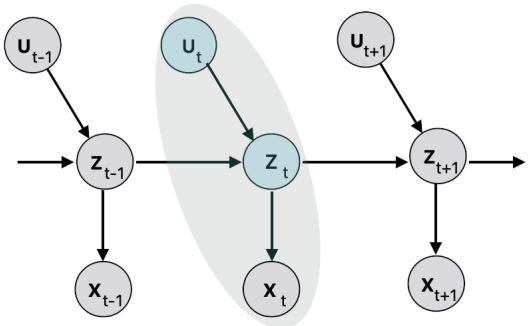


Figure 4.1: DKF model as a Bayesian network. The inputs $\mathbf{u}_t$ are always observed.

4.2.1 Generative model

The generative model is the joint distribution of observations and latent states given the inputs, $p(\mathbf{x}_{1:T}, \mathbf{z}_{1:T} \mid \mathbf{u}_{1:T})$. Ordering the variables in time and applying the product rule,

$$\begin{aligned} p(\mathbf{x}_{1:T}, \mathbf{z}_{1:T} \mid \mathbf{u}_{1:T}) &= \prod_{t=1}^{T} p(\mathbf{x}_t, \mathbf{z}_t \mid \mathbf{x}_{1:t-1}, \mathbf{z}_{1:t-1}, \mathbf{u}_{1:T}) \\ &= \prod_{t=1}^{T} p(\mathbf{x}_t \mid \mathbf{x}_{1:t-1}, \mathbf{z}_{1:t}, \mathbf{u}_{1:T})\, p(\mathbf{z}_t \mid \mathbf{x}_{1:t-1}, \mathbf{z}_{1:t-1}, \mathbf{u}_{1:T}). \end{aligned}$$

The graph of Figure 4.1 then simplifies each factor. Using d-separation (Annex I):

- $\mathbf{z}_t \perp\!\!\!\perp (\mathbf{x}_{1:t-1}, \mathbf{z}_{1:t-2}, \mathbf{u}_{1:t-1}, \mathbf{u}_{t+1:T}) \mid \mathbf{z}_{t-1}, \mathbf{u}_t$: every path from the past to $\mathbf{z}_t$ goes through $\mathbf{z}_{t-1}$, as a chain node or a fork node, and $\mathbf{z}_{t-1}$ is given. Future inputs only reach $\mathbf{z}_t$ through colliders on unobserved future states.
- $\mathbf{x}_t \perp\!\!\!\perp (\mathbf{x}_{1:t-1}, \mathbf{z}_{1:t-1}, \mathbf{u}_{1:T}) \mid \mathbf{z}_t$: $\mathbf{z}_t$ is the only parent of $\mathbf{x}_t$, and every path into $\mathbf{x}_t$ goes through it.

Hence[1]

$$p(\mathbf{x}_{1:T}, \mathbf{z}_{1:T} \mid \mathbf{u}_{1:T}) = \prod_{t=1}^{T} p(\mathbf{x}_t \mid \mathbf{z}_t)\, p(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{u}_t). \tag{4.3}$$

4.3 The posterior and the inference model

Learning a *causal* generative model does not require *causal* inference. To infer $\mathbf{z}_t$ we may use all observations, past and future, exactly as a Kalman smoother does.

4.3.1 The four inference models of the paper

Krishnan et al. [1] experiment with four variational models:

- **q-INDEP**: $q(\mathbf{z}_t \mid \mathbf{x}_t, \mathbf{u}_t)$, parameterised by a multi-layer perceptron (MLP);
- **q-LR**: $q(\mathbf{z}_t \mid \mathbf{x}_{t-1}, \mathbf{x}_t, \mathbf{x}_{t+1}, \mathbf{u}_{t-1}, \mathbf{u}_t, \mathbf{u}_{t+1})$, parameterised by an MLP;
- **q-RNN**: $q(\mathbf{z}_t \mid \mathbf{x}_{1:t}, \mathbf{u}_{1:t})$, parameterised by a recurrent neural network (RNN);
- **q-BRNN**: $q(\mathbf{z}_t \mid \mathbf{x}_{1:T}, \mathbf{u}_{1:T})$, parameterised by a bidirectional RNN.

None of these conditions on the previous latent state: the latent states are independent given the observations. Such *mean-field* approximations cannot represent the strong correlation between successive states of the true posterior. The same paper, however, writes the posterior in its structured form (its Equation 3), and the authors' follow-up work [4] builds an inference network of exactly that form, with a "combiner" that merges $\mathbf{z}_{t-1}$ with an RNN summary of the observations. My implementation follows the structured form, derived next.

4.3.2 Structure of the true posterior

By the product rule,

$$p_\theta(\mathbf{z}_{1:T} \mid \mathbf{x}_{1:T}, \mathbf{u}_{1:T}) = \prod_{t=1}^{T} p_\theta(\mathbf{z}_t \mid \mathbf{z}_{1:t-1}, \mathbf{x}_{1:T}, \mathbf{u}_{1:T}).$$

Given $\mathbf{z}_{t-1}$, the past variables $\mathbf{z}_{1:t-2}$, $\mathbf{x}_{1:t-1}$ and $\mathbf{u}_{1:t-1}$ are d-separated from $\mathbf{z}_t$: in the graph, every path from them to $\mathbf{z}_t$ passes through $\mathbf{z}_{t-1}$ as a chain node or a fork node, and observing $\mathbf{x}$ or $\mathbf{u}$ cannot open a path, because no $\mathbf{x}$ or $\mathbf{u}$ node is a collider on such a path. The future observations $\mathbf{x}_{t:T}$, on the other hand, remain dependent on $\mathbf{z}_t$. Therefore

$$p_\theta(\mathbf{z}_{1:T} \mid \mathbf{x}_{1:T}, \mathbf{u}_{1:T}) = \prod_{t=1}^{T} p_\theta\big(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{t:T}, \mathbf{u}_{t:T}\big),$$

[1] For $t = 1$, $p(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{u}_t)$ stands for $p(\mathbf{z}_1 \mid \mathbf{z}_0, \mathbf{u}_1)$, the prior on $\mathbf{z}_1$.

and it is natural to choose an inference model with the same structure:

$$q_\phi(\mathbf{z}_{1:T} \mid \mathbf{x}_{1:T}, \mathbf{u}_{1:T}) = \prod_{t=1}^{T} q_\phi\big(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{t:T}, \mathbf{u}_{t:T}\big). \tag{4.4}$$

In words

Once you know where the state was at time $t-1$, the past observations have nothing more to tell you about the state at time t: their information has been "absorbed" by $\mathbf{z}_{t-1}$. The future observations, however, still matter, because they were generated from states that $\mathbf{z}_t$ influences. So the posterior factor for $\mathbf{z}_t$ needs exactly two ingredients: the previous state and a summary of the observations from t to T. The inference network is built to receive exactly these two ingredients, and nothing else.

Notation for $t = 1$

To keep formulas short, I write $p_\theta(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \star)$ and $q_\phi(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \star)$ also for $t = 1$, where $\mathbf{z}_0$ is the fixed or learned initial value. Thus $p_\theta(\mathbf{z}_1 \mid \mathbf{z}_0, \star)$ is the prior on $\mathbf{z}_1$.

4.4 Evidence Lower Bound

The ELBO has the same definition as for the VAE, with the whole sequence in place of a single data point:

$$\mathcal{L}(\theta, \phi; \mathbf{x}_{1:T}, \mathbf{u}_{1:T}) = \mathbb{E}_{q_\phi(\mathbf{z}_{1:T} \mid \mathbf{x}_{1:T}, \mathbf{u}_{1:T})}\left[\log \frac{p_\theta(\mathbf{x}_{1:T}, \mathbf{z}_{1:T} \mid \mathbf{u}_{1:T})}{q_\phi(\mathbf{z}_{1:T} \mid \mathbf{x}_{1:T}, \mathbf{u}_{1:T})}\right] \;\le\; \log p_\theta(\mathbf{x}_{1:T} \mid \mathbf{u}_{1:T}),$$

and, as in (3.1), the gap is $\mathrm{KL}\big(q_\phi(\mathbf{z}_{1:T} \mid \mathbf{x}_{1:T}, \mathbf{u}_{1:T}) \,\|\, p_\theta(\mathbf{z}_{1:T} \mid \mathbf{x}_{1:T}, \mathbf{u}_{1:T})\big)$. Substituting the factorisations (4.3) and (4.4), Appendix B shows that

$$\begin{aligned}\mathcal{L}(\theta, \phi) = &\sum_{t=1}^{T} \mathbb{E}_{q_\phi(\mathbf{z}_t \mid \mathbf{x}_{1:T}, \mathbf{u}_{1:T})}\big[\log p_\theta(\mathbf{x}_t \mid \mathbf{z}_t)\big] \\ &- \sum_{t=1}^{T} \mathbb{E}_{q_\phi(\mathbf{z}_{t-1} \mid \mathbf{x}_{1:T}, \mathbf{u}_{1:T})}\Big[\mathrm{KL}\big(q_\phi(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{t:T}, \mathbf{u}_{t:T}) \,\big\|\, p_\theta(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{u}_t)\big)\Big]\end{aligned} \tag{4.5}$$

where, for $t = 1$, the outer expectation is trivial because $\mathbf{z}_0$ is fixed.

- The **reconstruction term** measures how well each observation is explained by its latent state.
- The **regularisation term** measures, at each time step, how far the inference network's distribution for $\mathbf{z}_t$ is from what the dynamics predict from $\mathbf{z}_{t-1}$.

In words

The KL divergence between two distributions over whole trajectories splits into one KL per time step: this is the *chain rule* of the KL divergence, and it holds because both the model and the inference network are Markov chains. Each per-step KL compares two Gaussians for $\mathbf{z}_t$ *given* $\mathbf{z}_{t-1}$, so it has a closed form, however complicated the neural networks are. The price is the outer expectation over $\mathbf{z}_{t-1}$, which Section 4.7 shows how to handle.

4.5 DKF Inference Model Design

The datasets used in this project have no control input, so from now on $\mathbf{u}$ is dropped:

$$q_\phi(\mathbf{z}_{1:T} \mid \mathbf{x}_{1:T}) = \prod_{t=1}^{T} q_\phi(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{t:T}).$$

Each factor is a Gaussian with diagonal covariance,

$$q_\phi(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{t:T}) = \mathcal{N}\big(\mathbf{z}_t;\ \boldsymbol{\mu}_\phi(\mathbf{z}_{t-1}, \mathbf{x}_{t:T}),\ \mathrm{diag}(\boldsymbol{\sigma}^2_\phi(\mathbf{z}_{t-1}, \mathbf{x}_{t:T}))\big),$$

whose parameters are computed by neural networks[2]. The dependence on the whole suffix $\mathbf{x}_{t:T}$ is handled by a **backward RNN** (an LSTM run from T down to 1, Appendix A): its output at time t, written $\mathbf{b}_t$, summarises $\mathbf{x}_{t:T}$ (Figure 4.2). At each time step, a **combiner** (Appendix E) merges $\mathbf{z}_{t-1}$ with $\mathbf{b}_t$, and an MLP, the **encoder**, maps the result to $\boldsymbol{\mu}_\phi$ and $\log \boldsymbol{\sigma}^2_\phi$.

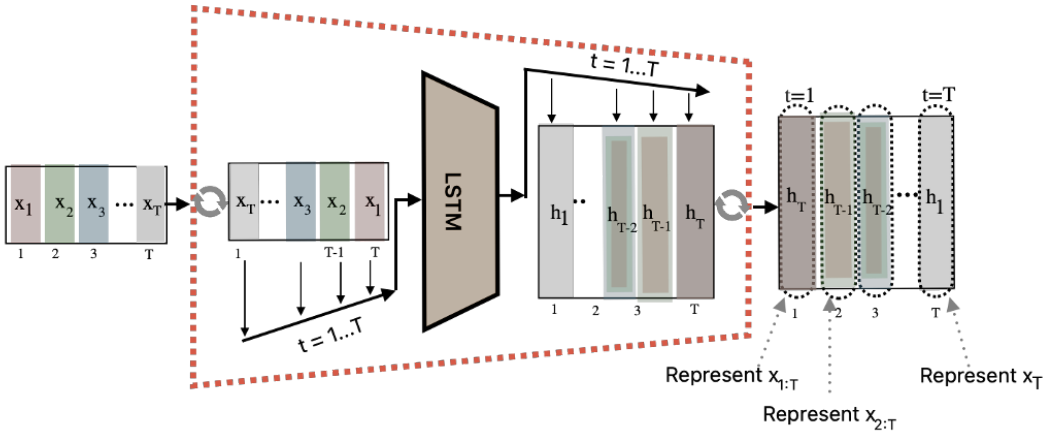


Figure 4.2: Backward RNN: the output at position t summarises $\mathbf{x}_{t:T}$.

4.5.1 Reparameterisation trick

As in the VAE, $\mathbf{z}_t$ is sampled as

$$\mathbf{z}_t = \boldsymbol{\mu}_\phi(\mathbf{z}_{t-1}, \mathbf{x}_{t:T}) + \boldsymbol{\sigma}_\phi(\mathbf{z}_{t-1}, \mathbf{x}_{t:T}) \odot \boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),$$

with $\boldsymbol{\sigma}_\phi$ the vector of standard deviations. Note that $\mathbf{z}_{t-1}$ is itself a sample, so $\mathbf{z}_t$ is a differentiable function of ϕ and of all the noises $\boldsymbol{\epsilon}_1, \ldots, \boldsymbol{\epsilon}_t$.

4.5.2 Overall design

The whole inference model is shown in Figure 4.3.

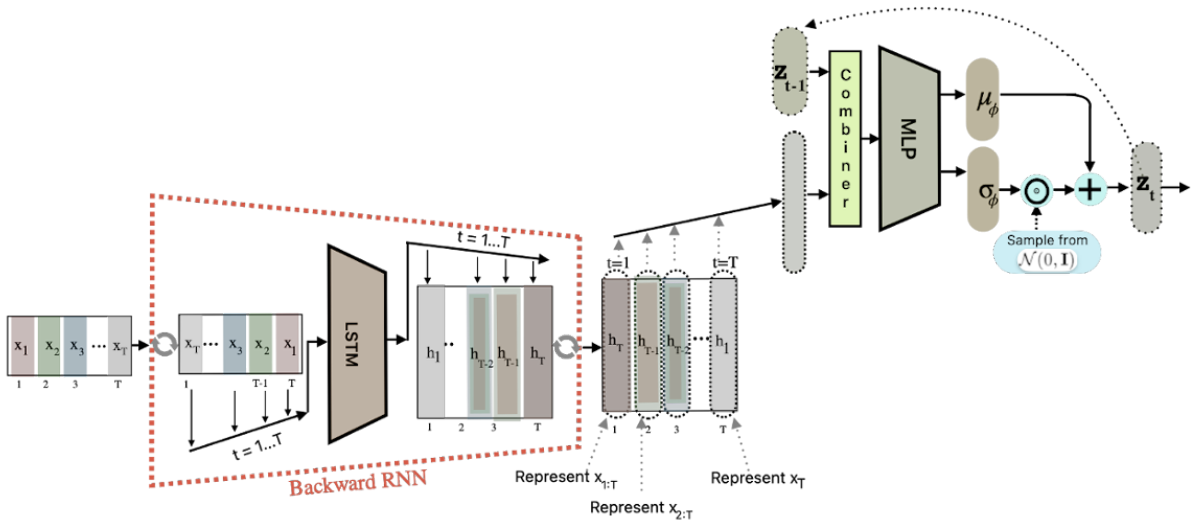


Figure 4.3: DKF inference model

[2]The networks output $\log \boldsymbol{\sigma}^2$ rather than $\boldsymbol{\sigma}^2$, so that the variance, obtained with an exponential, is always positive.

4.6 DKF Generative Model Design

Without control input, the generative model (4.3) reads

$$p_\theta(\mathbf{x}_{1:T}, \mathbf{z}_{1:T}) = \prod_{t=1}^{T} p_{\theta_\mathbf{x}}(\mathbf{x}_t \mid \mathbf{z}_t)\, p_{\theta_\mathbf{z}}(\mathbf{z}_t \mid \mathbf{z}_{t-1}), \tag{4.6}$$

with diagonal Gaussians for both factors.

Transition model.

$$p_{\theta_\mathbf{z}}(\mathbf{z}_t \mid \mathbf{z}_{t-1}) = \mathcal{N}\big(\mathbf{z}_t;\ \boldsymbol{\mu}_{\theta_\mathbf{z}}(\mathbf{z}_{t-1}),\ \mathrm{diag}(\boldsymbol{\sigma}^2_{\theta_\mathbf{z}}(\mathbf{z}_{t-1}))\big).$$

Emission (reconstruction) model.

$$p_{\theta_\mathbf{x}}(\mathbf{x}_t \mid \mathbf{z}_t) = \mathcal{N}\big(\mathbf{x}_t;\ \boldsymbol{\mu}_{\theta_\mathbf{x}}(\mathbf{z}_t),\ \mathrm{diag}(\boldsymbol{\sigma}^2_{\theta_\mathbf{x}}(\mathbf{z}_t))\big).$$

Both pairs of parameters are computed by MLPs (Figure 4.4), which again output $\log \boldsymbol{\sigma}^2$.

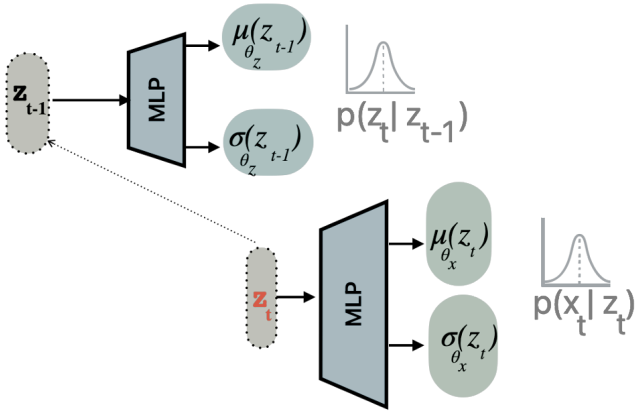


Figure 4.4: DKF generative model

Reconstruction

Given a latent state $\mathbf{z}_t$, the most likely observation is the mean $\hat{\mathbf{x}}_t = \boldsymbol{\mu}_{\theta_\mathbf{x}}(\mathbf{z}_t)$. This is what I call the "reconstruction" of $\mathbf{x}_t$ in the experiments.

4.7 Estimating the ELBO

4.7.1 The difficulty

Both terms of (4.5) are expectations under *marginals* of the inference model, $q_\phi(\mathbf{z}_t \mid \mathbf{x}_{1:T})$ and $q_\phi(\mathbf{z}_{t-1} \mid \mathbf{x}_{1:T})$. These marginals are intractable: for instance

$$q_\phi(\mathbf{z}_t \mid \mathbf{x}_{1:T}) = \int q_\phi(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{t:T})\, q_\phi(\mathbf{z}_{t-1} \mid \mathbf{x}_{1:T})\, d\mathbf{z}_{t-1}$$

is an integral over all the earlier latent states. With non-linear networks it is not Gaussian, and in general it has no closed form at all.

4.7.2 Ancestral sampling

We never need the *density* of these marginals, only *samples* from them. Draw one trajectory by ancestral sampling:

$$\dot{\mathbf{z}}_1 \sim q_\phi(\mathbf{z}_1 \mid \mathbf{z}_0, \mathbf{x}_{1:T}), \quad \dot{\mathbf{z}}_2 \sim q_\phi(\mathbf{z}_2 \mid \dot{\mathbf{z}}_1, \mathbf{x}_{2:T}), \quad \ldots, \quad \dot{\mathbf{z}}_T \sim q_\phi(\mathbf{z}_T \mid \dot{\mathbf{z}}_{T-1}, \mathbf{x}_T).$$

By (4.4), the trajectory $\dot{\mathbf{z}}_{1:T}$ is an exact sample from the joint $q_\phi(\mathbf{z}_{1:T} \mid \mathbf{x}_{1:T})$. Consequently, each $\dot{\mathbf{z}}_t$ is an exact sample from the marginal $q_\phi(\mathbf{z}_t \mid \mathbf{x}_{1:T})$. One trajectory provides samples of every marginal at once.

4.7.3 The estimator

Evaluate both terms of (4.5) along the sampled trajectory:

$$\hat{\mathcal{L}} = \sum_{t=1}^{T} \log p_\theta(\mathbf{x}_t \mid \dot{\mathbf{z}}_t) - \sum_{t=1}^{T} \mathrm{KL}\big(q_\phi(\mathbf{z}_t \mid \dot{\mathbf{z}}_{t-1}, \mathbf{x}_{t:T}) \,\big\|\, p_\theta(\mathbf{z}_t \mid \dot{\mathbf{z}}_{t-1})\big), \tag{4.7}$$

where each KL is computed in closed form (Appendix D) at the sampled $\dot{\mathbf{z}}_{t-1}$.

Proposition 1. $\mathbb{E}\big[\hat{\mathcal{L}}\big] = \mathcal{L}(\theta, \phi; \mathbf{x}_{1:T})$*: the single-trajectory estimator is unbiased.*

Proof. By linearity of expectation it is enough to treat each term. Since $\dot{\mathbf{z}}_t \sim q_\phi(\mathbf{z}_t \mid \mathbf{x}_{1:T})$, $\mathbb{E}[\log p_\theta(\mathbf{x}_t \mid \dot{\mathbf{z}}_t)] = \mathbb{E}_{q_\phi(\mathbf{z}_t \mid \mathbf{x}_{1:T})}[\log p_\theta(\mathbf{x}_t \mid \mathbf{z}_t)]$. Likewise, since $\dot{\mathbf{z}}_{t-1} \sim q_\phi(\mathbf{z}_{t-1} \mid \mathbf{x}_{1:T})$, the expectation of the t-th KL term is the expectation of the KL under $q_\phi(\mathbf{z}_{t-1} \mid \mathbf{x}_{1:T})$. These are exactly the terms of (4.5). □

In words

Each term of the ELBO is an average of something over the latent states. Averages can be estimated by plugging in samples, and nothing non-linear is applied to those averages afterwards. So one sampled trajectory gives an estimate that is right on average: some trajectories overshoot, some undershoot, and the errors cancel in expectation. That is what makes training with one trajectory per sequence legitimate. Averaging over the sequences of a minibatch then reduces the noise.

A numerical demonstration of Proposition 1, on a model where the ELBO can be computed exactly, is in the notebook `elbo_expectations.ipynb`.

4.7.4 Why compute the KL terms in closed form?

One could also estimate the regularisation term by sampling, using $\log q_\phi(\dot{\mathbf{z}}_t \mid \dot{\mathbf{z}}_{t-1}, \mathbf{x}_{t:T}) - \log p_\theta(\dot{\mathbf{z}}_t \mid \dot{\mathbf{z}}_{t-1})$ in place of each KL. That is also unbiased. Computing the KL in closed form instead integrates $\mathbf{z}_t$ out exactly given $\dot{\mathbf{z}}_{t-1}$. This is *Rao–Blackwellisation*, and by the law of total variance it lowers the variance of each KL term. Krishnan et al. report that sampling the KL term gives high-variance gradients, which motivates this choice.

Two remarks are worth making. First, the factorisation (4.5) is what makes a closed form possible at all: the KL between two distributions over whole trajectories has no closed form when the transition is a neural network, while each per-step KL is between two diagonal Gaussians. Second, lower variance for each term does not guarantee lower variance for the whole estimate, because the sampled terms are correlated. In the experiments of the notebook cited above, the closed-form KL gave less noisy gradients for every encoder tried, but the sampled version gave a less noisy *value* of the ELBO when the encoder was close to the true posterior.

4.7.5 Differentiability

With the reparameterisation trick, $\hat{\mathcal{L}}$ is a differentiable function of θ and ϕ for fixed noises $\boldsymbol{\epsilon}_{1:T}$, and its gradient is an unbiased estimate of $\nabla\mathcal{L}$. This is all stochastic gradient descent needs.

4.8 Training the DKF

The DKF is trained by minimising $-\hat{\mathcal{L}}$, averaged over a minibatch of sequences, with respect to $\theta_{\mathbf{x}}$, $\theta_{\mathbf{z}}$ and ϕ, using the Adam optimiser. The complete model is shown in Figure 4.5.

One training step

1. **Summarise the future.** Run the backward RNN over $\mathbf{x}_{1:T}$ to obtain $\mathbf{b}_1, \ldots, \mathbf{b}_T$, where $\mathbf{b}_t$ summarises $\mathbf{x}_{t:T}$.
2. **Sample a latent trajectory (ancestral sampling).** For $t = 1, \ldots, T$: combine $\dot{\mathbf{z}}_{t-1}$ with $\mathbf{b}_t$, compute $\boldsymbol{\mu}_\phi$ and $\log \boldsymbol{\sigma}^2_\phi$ with the encoder, and draw $\dot{\mathbf{z}}_t$ with the reparameterisation trick.

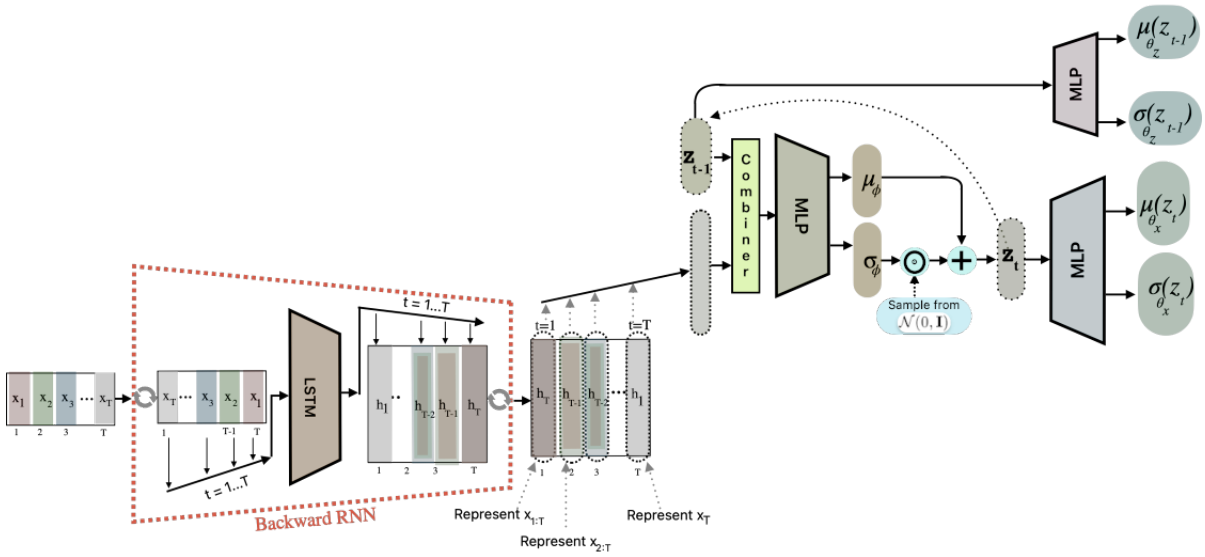


Figure 4.5: DKF model: inference network (left and centre) and generative networks (right).

3. **Evaluate the transition.** For each t, compute $\boldsymbol{\mu}_{\theta_\mathbf{z}}(\dot{\mathbf{z}}_{t-1})$ and $\log \boldsymbol{\sigma}^2_{\theta_\mathbf{z}}(\dot{\mathbf{z}}_{t-1})$ at the *sampled* previous state.
4. **Decode.** For each t, compute $\boldsymbol{\mu}_{\theta_\mathbf{x}}(\dot{\mathbf{z}}_t)$ and $\log \boldsymbol{\sigma}^2_{\theta_\mathbf{x}}(\dot{\mathbf{z}}_t)$.
5. **Compute the loss** $-\hat{\mathcal{L}}$ (Section 4.8.1).
6. **Update.** Backpropagate through the whole computation, including the sampling steps thanks to the reparameterisation trick, and take one optimiser step.

4.8.1 The loss function

Using (3) (Appendix C) and (4) (Appendix D), and writing d_x and d_z for the dimensions of $\mathbf{x}_t$ and $\mathbf{z}_t$,

$$-\sum_{t=1}^{T} \log p_{\theta_\mathbf{x}}(\mathbf{x}_t \mid \dot{\mathbf{z}}_t) = \frac{T d_x}{2}\log(2\pi) + \frac{1}{2}\sum_{t=1}^{T}\sum_{i=1}^{d_x}\left[\log \sigma^2_{\theta_\mathbf{x},i}(\dot{\mathbf{z}}_t) + \frac{\big(x_{t,i} - \mu_{\theta_\mathbf{x},i}(\dot{\mathbf{z}}_t)\big)^2}{\sigma^2_{\theta_\mathbf{x},i}(\dot{\mathbf{z}}_t)}\right],$$

$$\sum_{t=1}^{T} \mathrm{KL}_t = -\frac{T d_z}{2} + \frac{1}{2}\sum_{t=1}^{T}\sum_{i=1}^{d_z}\left[\frac{\sigma^2_{\phi,i} + (\mu_{\theta_\mathbf{z},i} - \mu_{\phi,i})^2}{\sigma^2_{\theta_\mathbf{z},i}} + \log \sigma^2_{\theta_\mathbf{z},i} - \log \sigma^2_{\phi,i}\right],$$

where, in the second line, the parameters of q_ϕ are evaluated at $(\dot{\mathbf{z}}_{t-1}, \mathbf{x}_{t:T})$ and those of $p_{\theta_\mathbf{z}}$ at $\dot{\mathbf{z}}_{t-1}$. The constants $\frac{T d_x}{2}\log(2\pi)$ and $-\frac{T d_z}{2}$ do not depend on the parameters and can be dropped. This leaves the two losses

$$\boxed{\mathcal{L}_{\text{rec}} = \frac{1}{2}\sum_{t=1}^{T}\sum_{i=1}^{d_x}\left[\log \sigma^2_{\theta_\mathbf{x},i} + \frac{(x_{t,i} - \mu_{\theta_\mathbf{x},i})^2}{\sigma^2_{\theta_\mathbf{x},i}}\right]}$$

$$\boxed{\mathcal{L}_{\text{reg}} = \frac{1}{2}\sum_{t=1}^{T}\sum_{i=1}^{d_z}\left[\frac{\sigma^2_{\phi,i} + (\mu_{\theta_\mathbf{z},i} - \mu_{\phi,i})^2}{\sigma^2_{\theta_\mathbf{z},i}} + \log \sigma^2_{\theta_\mathbf{z},i} - \log \sigma^2_{\phi,i}\right]}$$

and $-\hat{\mathcal{L}} = \mathcal{L}_{\text{rec}} + \mathcal{L}_{\text{reg}} + \text{const}$.

What the code computes

The function `loss_function` in `Utils/loss_fcts.py` computes $\frac{2}{T}\big(\mathcal{L}_{\text{rec}} + \beta\,\mathcal{L}_{\text{reg}}\big)$, averaged over the minibatch: the factor $\frac{1}{2}$ is omitted and the result is divided by the sequence length. Neither changes the minimiser. With $\beta = 1$ this is exactly the negative ELBO estimate (4.7), up to a constant and a positive

factor. With $\beta \neq 1$ the objective is a weighted (β-VAE-style) variant. The option `loss_type='mse'` drops the decoder variance, so its reconstruction term is no longer a log-likelihood and the objective is no longer an ELBO.

4.9 Implementation

The main classes of the implementation[3] are shown in Figure 4.6.

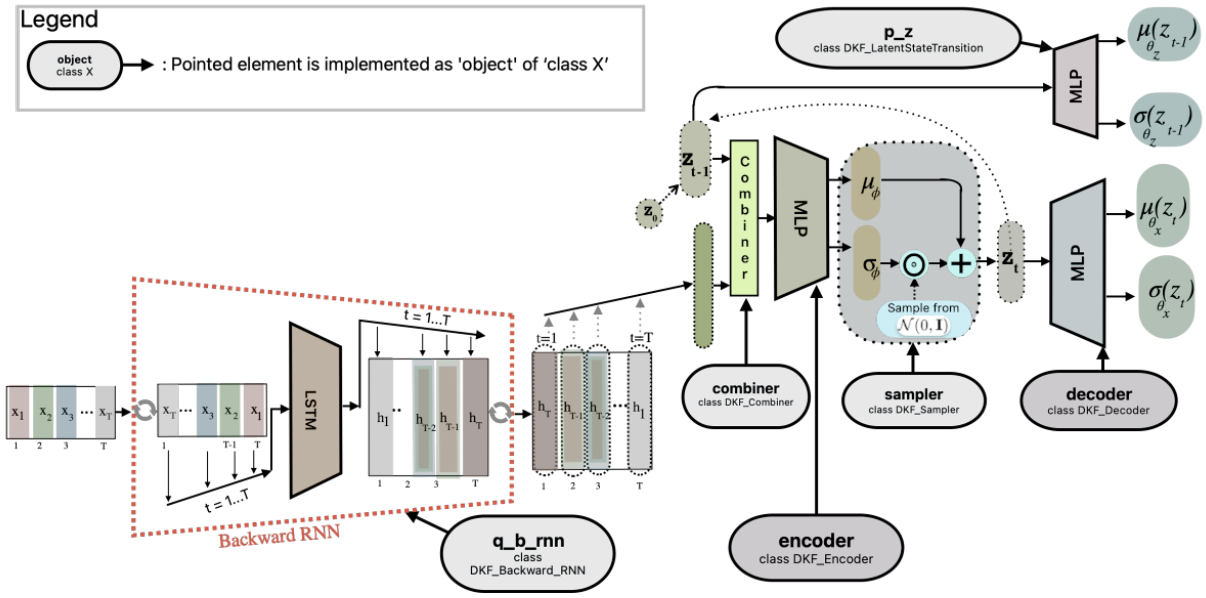


Figure 4.6: DKF implementation classes

4.9.1 The forward pass

The `forward` method of the `DKF` class implements steps 1 to 4 of the training procedure for a minibatch of sequences $\mathbf{x}$ of shape `(seq_len, batch_size, x_dim)`.

1. **Initial state.** $\mathbf{z}_0$ is the learnable parameter `Z_0` if `learnable_init_z=True`, and a tensor of zeros otherwise. It is expanded to the batch size.
2. **Backward RNN.** $\mathbf{x}$ is passed through the backward RNN `q_b_rnn`, which returns $\mathbf{b}_1, \ldots, \mathbf{b}_T$.
3. **Ancestral sampling.** For each t, the `combiner` merges the previous sample with $\mathbf{b}_t$, the `encoder` returns $\boldsymbol{\mu}_\phi$ and $\log \boldsymbol{\sigma}^2_\phi$, and the `sampler` draws $\dot{\mathbf{z}}_t$ with the reparameterisation trick. Means, log-variances and samples are stored.
4. **Transition.** The transition network `p_z` is applied to the shifted sequence $(\mathbf{z}_0, \dot{\mathbf{z}}_1, \ldots, \dot{\mathbf{z}}_{T-1})$, giving the parameters of $p(\mathbf{z}_t \mid \dot{\mathbf{z}}_{t-1})$ for every t.
5. **Decoding.** The `decoder` is applied to $\dot{\mathbf{z}}_{1:T}$ and returns $\hat{\mathbf{x}} = \boldsymbol{\mu}_{\theta_x}$ and $\log \boldsymbol{\sigma}^2_{\theta_x}$.

It returns everything the loss function needs.

4.9.2 The `predict` function

`predict` takes an observed sequence $\mathbf{x}_{1:T}$ and returns its reconstruction followed by `num_steps` forecasts.

1. It samples a latent trajectory $\dot{\mathbf{z}}_{1:T}$ exactly as in the forward pass, and decodes it to obtain the reconstruction.

[3] The code is available on GitHub.

2. Starting from $\dot{\mathbf{z}}_T$, it repeatedly applies the transition model: it computes the parameters of $p(\mathbf{z}_s \mid \mathbf{z}_{s-1})$, samples $\mathbf{z}_s$, and decodes it into the forecast $\boldsymbol{\mu}_{\theta_x}(\mathbf{z}_s)$.

Why this is the right way to forecast

At the last time step, the inference network conditions only on $\mathbf{z}_{T-1}$ and $\mathbf{x}_T$, so $q_\phi(\mathbf{z}_T \mid \mathbf{x}_{1:T})$ approximates the *filtering* distribution $p(\mathbf{z}_T \mid \mathbf{x}_{1:T})$: everything that has been observed, nothing more. Propagating samples of it through the learned dynamics is the non-linear analogue of the Kalman prediction step. Two limitations follow. A forecast is a single sampled path, not the mean of the predictive distribution, so its error includes the sampling noise. And the model can only forecast well if the transition network has learned the dynamics. A very flexible inference network can explain the training data while the transition network stays poor (Appendix F).

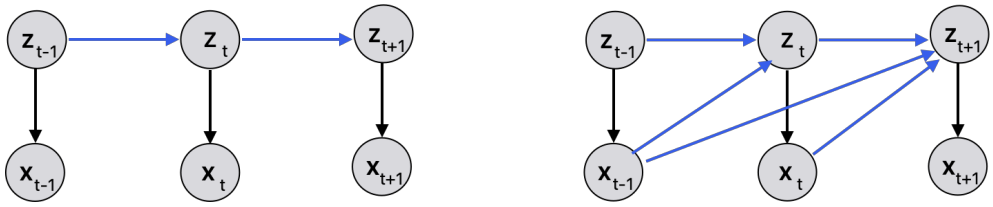
4.10 The Autoregressive DKF (AR DKF)

4.10.1 Motivation

On the sine-wave data, the vanilla DKF reconstructs the observed part of a sequence well but fails to forecast it (Appendix F). Its forecasts depend only on the transition model $p(\mathbf{z}_t \mid \mathbf{z}_{t-1})$, and nothing forces that model to carry the information needed for long-range prediction. To give the dynamics direct access to the history, I designed the "Autoregressive" DKF (AR DKF), whose transition also depends on the past observations:

$$p_{\theta_z}(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{0:t-1}),$$

where $\mathbf{x}_0$ is a fixed (zero) or learned initial value, so that the transition into $\mathbf{z}_1$ is defined. In other words, the independence $\mathbf{z}_t \perp\!\!\!\perp \mathbf{x}_{1:t-1} \mid \mathbf{z}_{t-1}$ of the vanilla DKF is given up on purpose by adding edges from the past observations to $\mathbf{z}_t$ (Figure 4.7).



(a) DKF model (b) AR DKF model

Figure 4.7: DKF model versus AR DKF model (blue: the added edges).

4.10.2 Generative model

The joint distribution becomes

$$p_\theta(\mathbf{x}_{1:T}, \mathbf{z}_{1:T}) = \prod_{t=1}^{T} p_{\theta_x}(\mathbf{x}_t \mid \mathbf{z}_t)\, p_{\theta_z}(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{0:t-1}). \tag{4.8}$$

It is still a valid, causal generative model: each variable depends only on variables generated before it. The history $\mathbf{x}_{0:t-1}$ is summarised by a **forward RNN** (`pz_f_rnn`) run on the shifted sequence $(\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_{T-1})$: its output $\mathbf{h}_{t-1}$ summarises $\mathbf{x}_{0:t-1}$, and an MLP maps $(\mathbf{z}_{t-1}, \mathbf{h}_{t-1})$ to the mean and log-variance of the transition. The ELBO keeps the form (4.5), with $p_{\theta_z}(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{0:t-1})$ in the KL terms, and it is estimated in the same way.

4.10.3 What the new edges change for inference

Changing the generative model changes its posterior, and therefore the inference network it needs. Redo the d-separation argument of Section 4.3 on the AR DKF graph.

Proposition 2. *In the AR DKF,*

$$p_\theta(\mathbf{z}_{1:T} \mid \mathbf{x}_{1:T}) = \prod_{t=1}^{T} p_\theta(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{1:T}),$$

and in general $p_\theta(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{1:T})$ depends on the past observations $\mathbf{x}_{1:t-1}$ as well as on $\mathbf{x}_{t:T}$.

Proof. Earlier states drop out. Take a path from $\mathbf{z}_s$, $s \leq t-2$, to $\mathbf{z}_t$. Paths through the latent chain pass through $\mathbf{z}_{t-1}$ as a chain or fork node and are blocked because $\mathbf{z}_{t-1}$ is given. Any other path must pass through an observation $\mathbf{x}_r$. Each observation has a single parent, $\mathbf{z}_r$, so it can never be a collider on a path: the path passes through $\mathbf{x}_r$ as a chain node ($\mathbf{z}_r \to \mathbf{x}_r \to \mathbf{z}_{r'}$) or as a fork node ($\mathbf{z}_{r'} \leftarrow \mathbf{x}_r \to \mathbf{z}_{r''}$). Since all observations are given, such a path is blocked too. *Past observations do not drop out.* For $t \geq 2$, $\mathbf{x}_{t-1}$ is a *parent* of $\mathbf{z}_t$: the edge $\mathbf{x}_{t-1} \to \mathbf{z}_t$ is a path of length one, with no intermediate node that could block it. Hence no conditioning set can make $\mathbf{z}_t$ independent of $\mathbf{x}_{t-1}$. □

In words

In the vanilla DKF, the past observations only influence $\mathbf{z}_t$ *through* $\mathbf{z}_{t-1}$, so knowing $\mathbf{z}_{t-1}$ makes them redundant. In the AR DKF they influence $\mathbf{z}_t$ *directly*, through the new edges, so they can never become redundant. The inference network for $\mathbf{z}_t$ must therefore see the past observations as well as the future ones.

The AR DKF as implemented keeps the inference network of the vanilla DKF, $q_\phi(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{t:T})$, which ignores $\mathbf{x}_{1:t-1}$. By Proposition 2 it cannot represent the true posterior, even with unlimited capacity, so the gap $\mathrm{KL}(q_\phi \,\|\, p_\theta(\cdot \mid \mathbf{x}))$ in the ELBO can never close. The variants of the next section repair this.

4.10.4 Prediction

Forecasting works as for the DKF, except that each transition step also needs the summary $\mathbf{h}$ of the observations so far. After a forecast $\hat{\mathbf{x}}_s$ is produced, it is appended to the sequence and the forward RNN is run again, so that the next transition step is conditioned on the model's own past forecasts. This makes the AR DKF a genuinely autoregressive forecaster.

A detail of the implementation

In `predict`, the sequence fed to the forward RNN is a sliding window: each new forecast is appended and the oldest observation is removed, and the RNN is re-run from the start of the window. During training, the RNN always starts from $\mathbf{x}_0$. The summaries seen at prediction time are therefore computed slightly differently from those seen during training.

The AR DKF design is shown in Figure 4.8.

4.11 The FAR DKF and BAR DKF models

The FAR DKF ("Forward Autoregressive") and the BAR DKF ("Bidirectional Autoregressive") keep the generative model of the AR DKF, including its transition $p_{\theta_z}(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{0:t-1})$, unchanged. They differ from it only in the **inference network**, which in both variants receives information about the past observations. By Proposition 2, this is exactly what the AR DKF posterior requires.

4.11.1 Forward Autoregressive DKF (FAR DKF)

The FAR DKF adds a second forward RNN, `q_f_rnn`, to the inference network (Figure 4.9). Its output at time t, $\overrightarrow{\mathbf{h}}_t$, summarises $\mathbf{x}_{1:t}$. The combiner merges $\dot{\mathbf{z}}_{t-1}$ with both $\mathbf{b}_t$ (the future) and $\overrightarrow{\mathbf{h}}_t$ (the past), so the inference model becomes

$$q_\phi(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{1:T}),$$

which has the structure of the true posterior of Proposition 2. The FAR DKF is therefore the principled inference model for the AR DKF generative model. The linear combiner offers separate weights for the two RNN summaries (Appendix E).

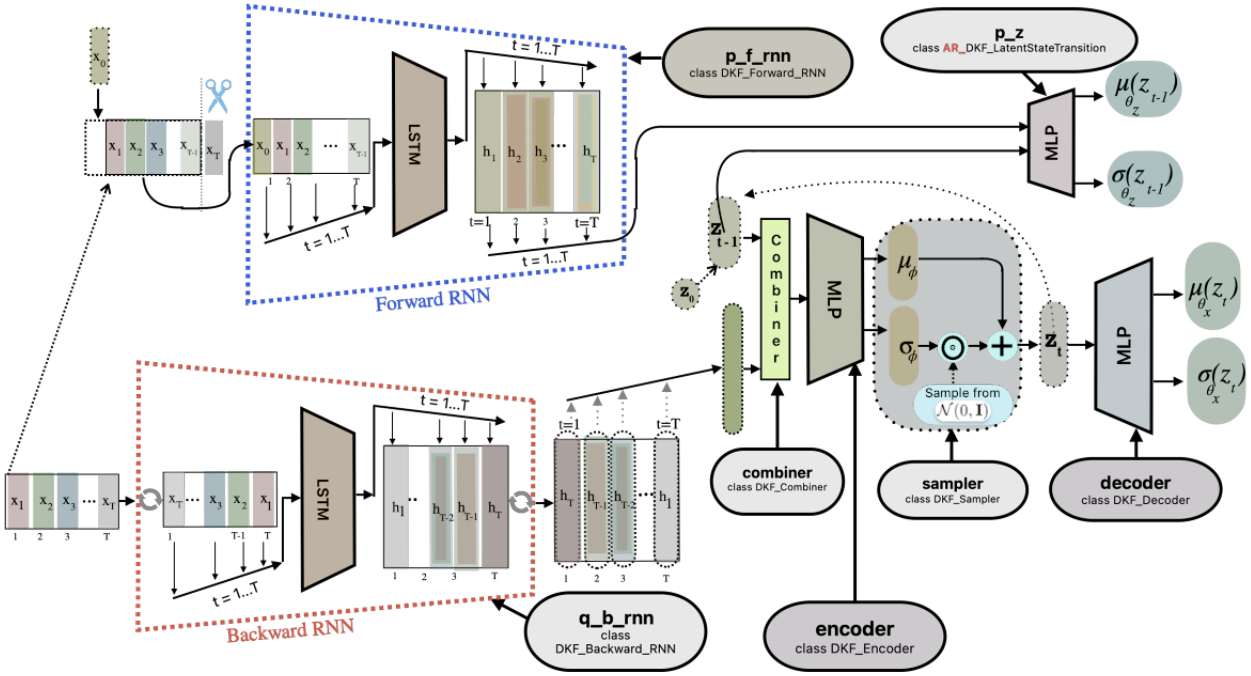


Figure 4.8: AR DKF model

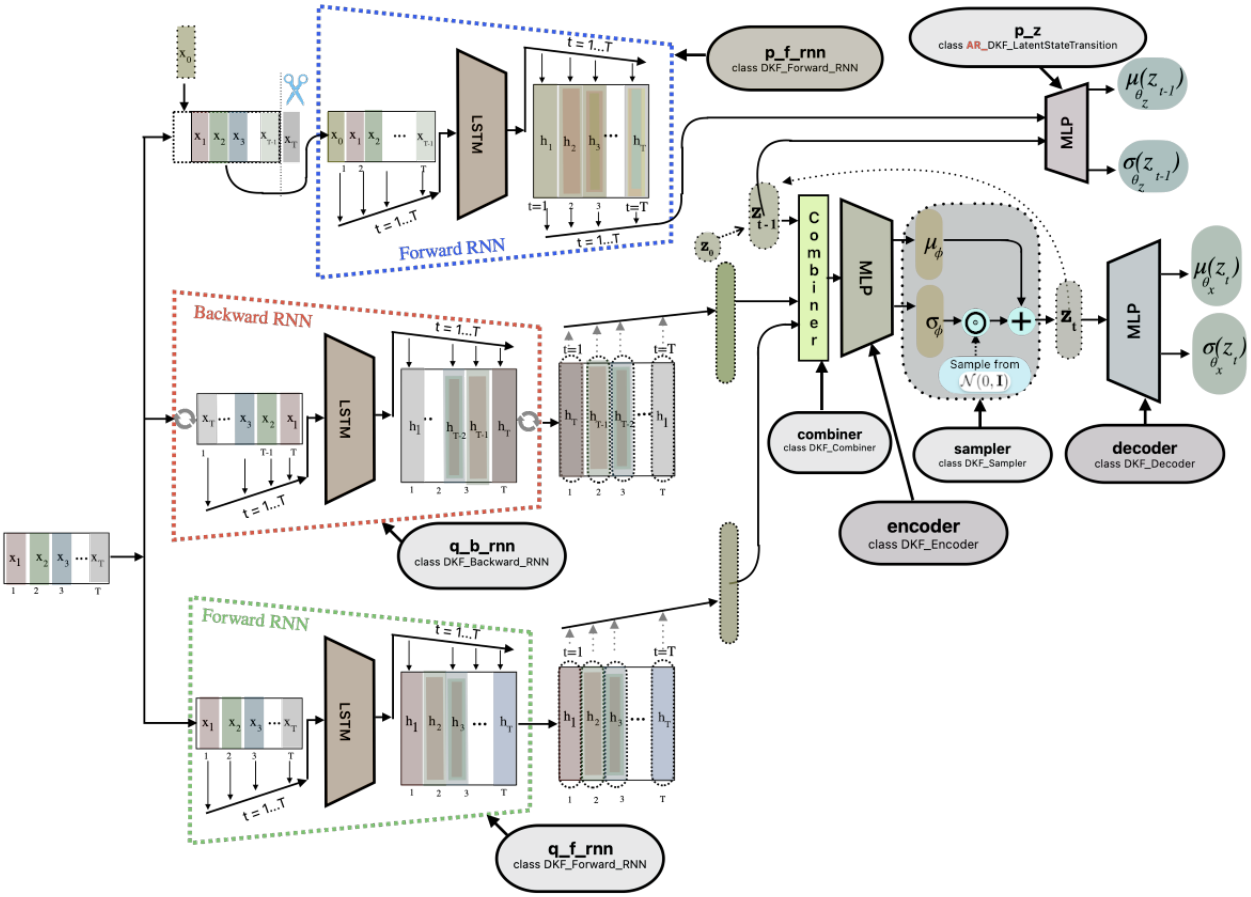


Figure 4.9: FAR DKF: the forward RNN `q_f_rnn` feeds the combiner of the inference network.

4.11.2 Bidirectional Autoregressive DKF (BAR DKF)

The BAR DKF replaces the backward RNN of the inference network by a single bidirectional RNN, whose output at time t combines a summary of $\mathbf{x}_{1:t}$ and a summary of $\mathbf{x}_{t:T}$. Its inference model therefore also has the form $q_\phi(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{1:T})$, with fewer parameters than the FAR DKF because the two directions are

merged before the combiner.

4.11.3 Summary

All three autoregressive variants share the generative model (4.8). They differ in their inference networks:

Model	**Inference network**	**Matches the posterior structure?**
AR DKF	$q_\phi(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{t:T})$	No: ignores $\mathbf{x}_{1:t-1}$
FAR DKF	$q_\phi(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{1:T})$, two RNNs	Yes
BAR DKF	$q_\phi(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{1:T})$, one bidirectional RNN	Yes

On the Lorenz data, over four training runs per experiment, the three variants reconstruct the observations equally well, and none of them forecasts consistently better than the others. Their ELBOs on test data vary more between runs of the same model than between models, and the estimated ELBO gaps are the same for the three variants (Appendix F). The advantage that Proposition 2 suggests for the FAR DKF and the BAR DKF is therefore not visible in these experiments.

Chapter 5

Conclusion

The Deep Kalman Filter keeps the probabilistic structure of the Kalman filter, a Markov chain of latent states with one emission per time step, and makes the transition and emission non-linear with neural networks. Exact inference is then impossible, and the model is trained like a VAE, by maximising a lower bound on the log-likelihood.

Three points that the original paper leaves implicit are central to understanding the method:

- **The structure of the inference network follows from d-separation.** Given $\mathbf{z}_{t-1}$, the true posterior of $\mathbf{z}_t$ depends only on the future observations $\mathbf{x}_{t:T}$, which is why the inference network combines $\mathbf{z}_{t-1}$ with a backward RNN.
- **The ELBO splits into one KL per time step**, by the chain rule of the KL divergence. Each of these is between two Gaussians given $\mathbf{z}_{t-1}$ and has a closed form.
- **One sampled trajectory gives an unbiased estimate of the ELBO.** The ELBO is linear in its expectations under the intractable marginals of the inference network, and ancestral sampling provides exact samples of all of these marginals at once.

The autoregressive variants I designed add edges from past observations to the latent state, which gives the transition model direct access to the history and greatly improves forecasting on the sine-wave data. Redoing the d-separation argument on the new graph shows that these edges also change the posterior: it now depends on the past observations as well as the future ones. The FAR DKF, whose inference network sees both, is therefore the principled inference model for the AR DKF generative model. On the Lorenz data, however, the three autoregressive variants perform comparably: over four training runs per experiment, neither the forecasting errors nor the ELBOs on test data rank them consistently, and the variability between runs is larger than the differences between models.

Several directions remain open. The variability between training runs should be reduced, or at least controlled with fixed random seeds and more repetitions, before the variants can be compared. Tighter estimates of the log-likelihood, with many more importance samples, would be needed to measure the differences between their ELBO gaps. Forecasts should be evaluated with the predictive mean, or with a probabilistic score, rather than with a single sampled path. The reconstruction error currently matches the noise level, which suggests that the inference network reproduces the observations, noise included, rather than denoising them: monitoring the two terms of the ELBO separately would show how much of the ELBO each part of the model accounts for. Finally, control inputs, for example from an inertial measurement unit, could be added to the transition model as in the original formulation.

Appendices

A Understanding the Backward LSTM

The inference network needs, at each time step t, a summary of the future observations $\mathbf{x}_{t:T}$. A backward RNN provides all of them in a single pass.

Definition

A backward LSTM is an ordinary LSTM run from $t = T$ down to $t = 1$. Its state at time t is computed from the observation $\mathbf{x}_t$ and the state at time $t+1$:

$$
\begin{aligned}
\mathbf{f}_t &= \sigma(\mathbf{W}_f[\mathbf{b}_{t+1}, \mathbf{x}_t] + \mathbf{c}_f) && \text{(forget gate)} \\
\mathbf{i}_t &= \sigma(\mathbf{W}_i[\mathbf{b}_{t+1}, \mathbf{x}_t] + \mathbf{c}_i) && \text{(input gate)} \\
\tilde{\mathbf{m}}_t &= \tanh(\mathbf{W}_m[\mathbf{b}_{t+1}, \mathbf{x}_t] + \mathbf{c}_m) && \text{(candidate memory)} \\
\mathbf{m}_t &= \mathbf{f}_t \odot \mathbf{m}_{t+1} + \mathbf{i}_t \odot \tilde{\mathbf{m}}_t && \text{(memory cell)} \\
\mathbf{o}_t &= \sigma(\mathbf{W}_o[\mathbf{b}_{t+1}, \mathbf{x}_t] + \mathbf{c}_o) && \text{(output gate)} \\
\mathbf{b}_t &= \mathbf{o}_t \odot \tanh(\mathbf{m}_t) && \text{(output)}
\end{aligned}
$$

with initial values $\mathbf{b}_{T+1} = \mathbf{0}$ and $\mathbf{m}_{T+1} = \mathbf{0}$. Here σ is the logistic sigmoid, $[\cdot, \cdot]$ denotes concatenation, and the $\mathbf{W}$'s and $\mathbf{c}$'s are learned weights and biases.

In words

Unrolling the recursion, $\mathbf{b}_t$ is a function of $\mathbf{x}_t, \mathbf{x}_{t+1}, \ldots, \mathbf{x}_T$ only. So $\mathbf{b}_t$ summarises exactly the suffix $\mathbf{x}_{t:T}$ that the posterior factor $q_\phi(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{t:T})$ needs. At $t = T$ it has seen only $\mathbf{x}_T$, and at $t = 1$ it has seen the whole sequence.

Implementation by flipping

Deep-learning libraries implement LSTMs that run forwards in time. The backward LSTM is obtained by flipping the sequence, running a forward LSTM, and flipping the outputs back:

1. flip: $(\mathbf{x}_T, \mathbf{x}_{T-1}, \ldots, \mathbf{x}_1)$;
2. run the LSTM: its k-th output, $k = 1, \ldots, T$, has seen $\mathbf{x}_T, \ldots, \mathbf{x}_{T-k+1}$;
3. flip the outputs back: the output now stored at position $t = T - k + 1$ has seen $\mathbf{x}_{t:T}$, i.e. it is $\mathbf{b}_t$.

This is what `DKF_Backward_RNN` does, with `torch.flip` along the time dimension (dimension 0, since tensors have shape `(seq_len, batch_size, dim)`).

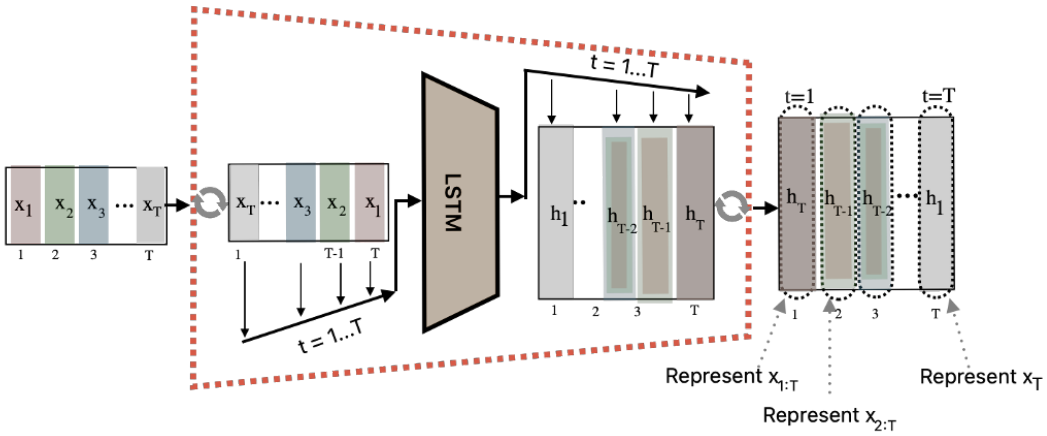


Figure 1: Backward LSTM. In this figure the hidden states are indexed by their position in the *flipped* sequence: h_k is $\mathbf{b}_{T-k+1}$ in the notation of this appendix.

B Detailed Derivation of the ELBO

Notation

As in the main text, $p_\theta(\mathbf{z}_1 \mid \mathbf{z}_0, \star)$ is the prior on $\mathbf{z}_1$, and $q_\phi(\mathbf{z}_1 \mid \mathbf{z}_0, \star)$ is the first factor of the inference network, with $\mathbf{z}_0$ fixed or learned. To lighten the formulas, q stands for $q_\phi(\cdot \mid \mathbf{x}_{1:T}, \mathbf{u}_{1:T})$ and its factors, and p for p_θ.

Step 1: the bound

As for the VAE (Chapter 3), Jensen's inequality gives

$$\log p(\mathbf{x}_{1:T} \mid \mathbf{u}_{1:T}) \;\geq\; \mathcal{L} := \mathbb{E}_{q(\mathbf{z}_{1:T})}\left[\log \frac{p(\mathbf{x}_{1:T}, \mathbf{z}_{1:T} \mid \mathbf{u}_{1:T})}{q(\mathbf{z}_{1:T})}\right],$$

with a gap equal to $\mathrm{KL}\big(q(\mathbf{z}_{1:T}) \,\|\, p(\mathbf{z}_{1:T} \mid \mathbf{x}_{1:T}, \mathbf{u}_{1:T})\big)$.

Step 2: substitute the factorisations

With the generative model (4.3) and the inference model (4.4),

$$\log \frac{p(\mathbf{x}_{1:T}, \mathbf{z}_{1:T} \mid \mathbf{u}_{1:T})}{q(\mathbf{z}_{1:T})} = \sum_{t=1}^{T} \log p(\mathbf{x}_t \mid \mathbf{z}_t) \;-\; \sum_{t=1}^{T} \underbrace{\log \frac{q(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{t:T}, \mathbf{u}_{t:T})}{p(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{u}_t)}}_{=:\, r_t(\mathbf{z}_{t-1}, \mathbf{z}_t)}.$$

By linearity of expectation,

$$\mathcal{L} = \sum_{t=1}^{T} \mathbb{E}_{q(\mathbf{z}_{1:T})}\big[\log p(\mathbf{x}_t \mid \mathbf{z}_t)\big] - \sum_{t=1}^{T} \mathbb{E}_{q(\mathbf{z}_{1:T})}\big[r_t(\mathbf{z}_{t-1}, \mathbf{z}_t)\big]. \tag{1}$$

Each term depends on one or two consecutive latent states only. The next step shows how to integrate out all the others.

Step 3: integrating out the latent states a term does not depend on

Proposition 3. *For any functions f and g,*

$$\begin{aligned}
\mathbb{E}_{q(\mathbf{z}_{1:T})}\big[f(\mathbf{z}_t)\big] &= \mathbb{E}_{q(\mathbf{z}_t)}\big[f(\mathbf{z}_t)\big], \\
\mathbb{E}_{q(\mathbf{z}_{1:T})}\big[g(\mathbf{z}_{t-1}, \mathbf{z}_t)\big] &= \mathbb{E}_{q(\mathbf{z}_{t-1})}\Big[\mathbb{E}_{q(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{t:T}, \mathbf{u}_{t:T})}\big[g(\mathbf{z}_{t-1}, \mathbf{z}_t)\big]\Big],
\end{aligned}$$

where $q(\mathbf{z}_t)$ and $q(\mathbf{z}_{t-1})$ denote the marginals of the inference model.

Proof. Write the expectation as an integral against $q(\mathbf{z}_{1:T}) = \prod_{s=1}^{T} q(\mathbf{z}_s \mid \mathbf{z}_{s-1}, \mathbf{x}_{s:T}, \mathbf{u}_{s:T})$.

Future states. The integrand does not depend on $\mathbf{z}_T$ (when $t < T$), and $\mathbf{z}_T$ appears only in the last factor, which is a probability density in $\mathbf{z}_T$. Hence

$$\int q(\mathbf{z}_T \mid \mathbf{z}_{T-1}, \mathbf{x}_T, \mathbf{u}_T)\, d\mathbf{z}_T = 1,$$

and $\mathbf{z}_T$ disappears. Repeating the argument for $\mathbf{z}_{T-1}, \dots, \mathbf{z}_{t+1}$ removes all future states. What remains is the integral against $q(\mathbf{z}_{1:t}) = \prod_{s=1}^{t} q(\mathbf{z}_s \mid \mathbf{z}_{s-1}, \cdot)$, the marginal of the first t states.

Past states. Split $q(\mathbf{z}_{1:t}) = q(\mathbf{z}_{1:t-1})\, q(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{t:T}, \mathbf{u}_{t:T})$. The inner integral over $\mathbf{z}_t$ is the inner expectation in the statement. The resulting function of $\mathbf{z}_{t-1}$ is then integrated against $q(\mathbf{z}_{1:t-1})$. Integrating out $\mathbf{z}_{1:t-2}$, on which it does not depend, gives the marginal $q(\mathbf{z}_{t-1})$ by definition of a marginal. For $f(\mathbf{z}_t)$, integrating out $\mathbf{z}_{1:t-1}$ in the same way gives $q(\mathbf{z}_t)$. □

About the factor $q(\mathbf{z}_t \mid \mathbf{z}_{1:t-1}, \cdot)$

The proof uses $q(\mathbf{z}_t \mid \mathbf{z}_{1:t-1}, \mathbf{x}_{1:T}, \mathbf{u}_{1:T}) = q(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{t:T}, \mathbf{u}_{t:T})$. This holds *by construction* of the inference model (4.4): the inference network is built to receive only these inputs. D-separation is what justified that design, because the true posterior has the same property; but in the derivation itself, the equality is a property of q, not a d-separation statement.

Step 4: the result

Apply Proposition 3 to (1), with $f = \log p(\mathbf{x}_t \mid \cdot)$ and $g = r_t$. For the second term, the inner expectation is a KL divergence:

$$\mathbb{E}_{q(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{t:T}, \mathbf{u}_{t:T})}\big[r_t(\mathbf{z}_{t-1}, \mathbf{z}_t)\big] = \mathrm{KL}\big(q(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{t:T}, \mathbf{u}_{t:T}) \,\big\|\, p(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{u}_t)\big).$$

Therefore

$$\boxed{\begin{aligned}\mathcal{L} = &\sum_{t=1}^{T} \mathbb{E}_{q_\phi(\mathbf{z}_t \mid \mathbf{x}_{1:T}, \mathbf{u}_{1:T})}\big[\log p_\theta(\mathbf{x}_t \mid \mathbf{z}_t)\big] \\ &- \sum_{t=1}^{T} \mathbb{E}_{q_\phi(\mathbf{z}_{t-1} \mid \mathbf{x}_{1:T}, \mathbf{u}_{1:T})}\Big[\mathrm{KL}\big(q_\phi(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_{t:T}, \mathbf{u}_{t:T}) \,\big\|\, p_\theta(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{u}_t)\big)\Big],\end{aligned}}$$

which is (4.5). The second sum is the *chain rule* of the KL divergence: for two Markov chains,

$$\mathrm{KL}\big(q(\mathbf{z}_{1:T}) \,\|\, p(\mathbf{z}_{1:T})\big) = \sum_{t=1}^{T} \mathbb{E}_{q(\mathbf{z}_{t-1})}\Big[\mathrm{KL}\big(q(\mathbf{z}_t \mid \mathbf{z}_{t-1}) \,\|\, p(\mathbf{z}_t \mid \mathbf{z}_{t-1})\big)\Big].$$

Remark: iterated expectations

Applying the proof of Proposition 3 to all time steps writes the expectation under $q(\mathbf{z}_{1:T})$ as *iterated* conditional expectations,

$$\mathbb{E}_{q(\mathbf{z}_{1:T})}[h] = \mathbb{E}_{q(\mathbf{z}_1 \mid \mathbf{z}_0, \cdot)}\Big[\mathbb{E}_{q(\mathbf{z}_2 \mid \mathbf{z}_1, \cdot)}\big[\cdots \mathbb{E}_{q(\mathbf{z}_T \mid \mathbf{z}_{T-1}, \cdot)}[h] \cdots\big]\Big].$$

Each inner expectation is passed to the next level as it is, with no non-linear function applied to it, so the iterated expectations collapse into a single expectation over the joint distribution (the tower property). That is why ancestral sampling, one sample per level, gives an unbiased estimate (Proposition 1).

C Detailed Mathematical Development of the Reconstruction Term

Problem Setup

Given the probability distribution:

$$p_{\theta_{\boldsymbol{x}}}(\boldsymbol{x}_t|\boldsymbol{z}_t) = \mathcal{N}(\boldsymbol{x}_t; \mu_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t), \Sigma^2_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t))$$

where $\Sigma^2_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t) = diag(\sigma^2_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t))$, we aim to develop the expression:

$$\sum_{t=1}^{T} \Big[\log p_\theta(x_t|z_t) \Big]$$

Logarithm of the Probability Distribution

The probability density function (PDF) of a multivariate normal distribution $\mathcal{N}(\boldsymbol{x}_t; \mu_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t), \Sigma^2_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t))$ is given by:

$$p_{\theta_{\boldsymbol{x}}}(\boldsymbol{x}_t|\boldsymbol{z}_t) = \frac{1}{(2\pi)^{d/2}\left|\Sigma^2_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t)\right|^{1/2}} \exp\left(-\frac{1}{2}\big(\boldsymbol{x}_t - \mu_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t)\big)^\top \big(\Sigma^2_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t)\big)^{-1}\big(\boldsymbol{x}_t - \mu_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t)\big)\right)$$

In this context, d represents the dimensionality of the random variable $\boldsymbol{x}_t$. Specifically, $\boldsymbol{x}_t$ is a d-dimensional vector, and $\mu_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t)$ is also a d-dimensional vector. The covariance matrix $\Sigma^2_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t)$ is a $d \times d$ diagonal matrix, where each diagonal element corresponds to the variance of one dimension of $\boldsymbol{x}_t$.

Taking the logarithm, we get:

$$\log p_{\theta_{\boldsymbol{x}}}(\boldsymbol{x}_t|\boldsymbol{z}_t) = -\frac{d}{2}\log(2\pi) - \frac{1}{2}\log\left|\Sigma^2_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t)\right| - \frac{1}{2}\big(\boldsymbol{x}_t - \mu_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t)\big)^\top \big(\Sigma^2_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t)\big)^{-1}\big(\boldsymbol{x}_t - \mu_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t)\big)$$

Simplify the Logarithm Expression

Given that $\Sigma^2_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t) = diag(\sigma^2_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t))$, we have:

$$\log|\Sigma^2_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t)| = \log\prod_{i=1}^{d}\sigma^2_{\theta_{\boldsymbol{x}},i}(\boldsymbol{z}_t) = \sum_{i=1}^{d}\log\sigma^2_{\theta_{\boldsymbol{x}},i}(\boldsymbol{z}_t)$$

and

$$(\Sigma^2_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t))^{-1} = \begin{pmatrix} \frac{1}{\sigma^2_{\theta_{\boldsymbol{x}},1}(\boldsymbol{z}_t)} & 0 & \cdots & 0 \\ 0 & \frac{1}{\sigma^2_{\theta_{\boldsymbol{x}},2}(\boldsymbol{z}_t)} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \frac{1}{\sigma^2_{\theta_{\boldsymbol{x}},d}(\boldsymbol{z}_t)} \end{pmatrix}$$

Thus,

$$\big(\boldsymbol{x}_t - \mu_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t)\big)^\top \big(\Sigma^2_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t)\big)^{-1}\big(\boldsymbol{x}_t - \mu_{\theta_{\boldsymbol{x}}}(\boldsymbol{z}_t)\big) = \sum_{i=1}^{d}\frac{\big(x_{t,i} - \mu_{\theta_{\boldsymbol{x}},i}(\boldsymbol{z}_t)\big)^2}{\sigma^2_{\theta_{\boldsymbol{x}},i}(\boldsymbol{z}_t)}$$

and

$$\log p_{\theta_{\boldsymbol{x}}}(\boldsymbol{x}_t|\boldsymbol{z}_t) = -\frac{d}{2}\log(2\pi) - \frac{1}{2}\sum_{i=1}^{d}\log\sigma^2_{\theta_{\boldsymbol{x}},i}(\boldsymbol{z}_t) - \frac{1}{2}\sum_{i=1}^{d}\frac{\big(x_{t,i} - \mu_{\theta_{\boldsymbol{x}},i}(\boldsymbol{z}_t)\big)^2}{\sigma^2_{\theta_{\boldsymbol{x}},i}(\boldsymbol{z}_t)} \tag{2}$$

Summing Over Time Steps

Now, we sum over all time steps t from 1 to T:

$$\sum_{t=1}^{T}\log p_{\theta_{\boldsymbol{x}}}(\boldsymbol{x}_t|\boldsymbol{z}_t) = \sum_{t=1}^{T}\left[-\frac{d}{2}\log(2\pi) - \frac{1}{2}\sum_{i=1}^{d}\log\sigma^2_{\theta_{\boldsymbol{x}},i}(\boldsymbol{z}_t) - \frac{1}{2}\sum_{i=1}^{d}\frac{\big(x_{t,i} - \mu_{\theta_{\boldsymbol{x}},i}(\boldsymbol{z}_t)\big)^2}{\sigma^2_{\theta_{\boldsymbol{x}},i}(\boldsymbol{z}_t)}\right]$$

Final Expression

$$\sum_{t=1}^{T} \log p_{\theta_{\boldsymbol{x}}}(\boldsymbol{x}_t|\boldsymbol{z}_t) = -\frac{Td}{2}\log(2\pi) - \frac{1}{2}\sum_{t=1}^{T}\sum_{i=1}^{d}\left[\log \sigma^2_{\theta_{\boldsymbol{x}},i}(\boldsymbol{z}_t) + \frac{\left(x_{t,i} - \mu_{\theta_{\boldsymbol{x}},i}(\boldsymbol{z}_t)\right)^2}{\sigma^2_{\theta_{\boldsymbol{x}},i}(\boldsymbol{z}_t)}\right] \tag{3}$$

In the estimator (4.7), this expression is evaluated at the sampled latent states $\dot{\boldsymbol{z}}_{1:T}$. The first term, $-\frac{Td}{2}\log(2\pi)$, does not depend on the parameters and is dropped from the loss (Section 4.8.1).

D Detailed Mathematical Development of the Regularization Term

(Computation is done here without the control input term $\boldsymbol{u}$.)

Problem Setup

Given the following distributions:

$$p_{\theta_{\boldsymbol{z}}}(\boldsymbol{z}_t|\boldsymbol{z}_{t-1}) = \mathcal{N}(\boldsymbol{z}_t; \mu_{\theta_{\boldsymbol{z}}}(\boldsymbol{z}_{t-1}), \Sigma^2_{\theta_{\boldsymbol{z}}}(\boldsymbol{z}_{t-1})) = \mathcal{N}(\boldsymbol{z}_t; \mu_{\theta_{\boldsymbol{z}}}(\boldsymbol{z}_{t-1}), \mathrm{diag}(\sigma^2_{\theta_{\boldsymbol{z}}}(\boldsymbol{z}_{t-1})))$$

and

$$q_\phi(\boldsymbol{z}_t|\boldsymbol{z}_{t-1}, \boldsymbol{x}_{t:T}) = \mathcal{N}(\boldsymbol{z}_t; \mu_\phi(\boldsymbol{z}_{t-1}, \boldsymbol{x}_{t:T}), \Sigma^2_\phi(\boldsymbol{z}_{t-1}, \boldsymbol{x}_{t:T})) = \mathcal{N}(\boldsymbol{z}_t; \mu_\phi(\boldsymbol{z}_{t-1}, \boldsymbol{x}_{t:T}), \mathrm{diag}(\sigma^2_\phi(\boldsymbol{z}_{t-1}, \boldsymbol{x}_{t:T})))$$

We aim to compute:

$$\sum_{t=1}^{T} \mathrm{KL}\big(q_\phi(z_t|z_{t-1}, x_{t:T}) \parallel p_\theta(z_t|z_{t-1})\big)$$

KL Divergence Between Two Gaussian Distributions

The KL divergence between two multivariate Gaussian distributions $\mathcal{N}(\boldsymbol{z}; \boldsymbol{\mu}_1, \Sigma_1)$ and $\mathcal{N}(\boldsymbol{z}; \boldsymbol{\mu}_2, \Sigma_2)$ is given by:

$$\mathrm{KL}\big(\mathcal{N}(\boldsymbol{z}; \boldsymbol{\mu}_1, \Sigma_1) \parallel \mathcal{N}(\boldsymbol{z}; \boldsymbol{\mu}_2, \Sigma_2)\big) = \frac{1}{2}\left(\mathrm{tr}(\Sigma_2^{-1}\Sigma_1) + (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^T\Sigma_2^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) - k + \log\frac{|\Sigma_2|}{|\Sigma_1|}\right)$$

where k is the dimensionality of the Gaussian distributions.

Simplification for Diagonal Covariance Matrices

Since both q_ϕ and p_θ are diagonal Gaussian distributions, their covariance matrices are diagonal. Let's denote:

$$\Sigma_\phi = \mathrm{diag}\big(\sigma^2_\phi(\boldsymbol{z}_{t-1}, \boldsymbol{x}_{t:T})\big)$$
$$\Sigma_\theta = \mathrm{diag}\big(\sigma^2_{\theta_{\boldsymbol{z}}}(\boldsymbol{z}_{t-1})\big)$$

For diagonal matrices, the trace and determinant operations simplify:

$$\mathrm{tr}(\Sigma_\theta^{-1}\Sigma_\phi) = \sum_i \frac{\sigma^2_{\phi,i}(\boldsymbol{z}_{t-1}, \boldsymbol{x}_{t:T})}{\sigma^2_{\theta_{\boldsymbol{z}},i}(\boldsymbol{z}_{t-1})}$$

$$\log\frac{|\Sigma_\theta|}{|\Sigma_\phi|} = \sum_i \log\frac{\sigma^2_{\theta_{\boldsymbol{z}},i}(\boldsymbol{z}_{t-1})}{\sigma^2_{\phi,i}(\boldsymbol{z}_{t-1}, \boldsymbol{x}_{t:T})}$$

Final Computation

The KL divergence for each time step t is:

$$\begin{aligned}\mathrm{KL}\big(q_\phi(\boldsymbol{z}_t|\boldsymbol{z}_{t-1}, \boldsymbol{x}_{t:T}) \parallel p_\theta(\boldsymbol{z}_t|\boldsymbol{z}_{t-1})\big) = \frac{1}{2}\Bigg(&\sum_i \frac{\sigma^2_{\phi,i}(\boldsymbol{z}_{t-1}, \boldsymbol{x}_{t:T})}{\sigma^2_{\theta_{\boldsymbol{z}},i}(\boldsymbol{z}_{t-1})} \quad + \\ &\sum_i (\mu_{\theta_{\boldsymbol{z}},i}(\boldsymbol{z}_{t-1}) - \mu_{\phi,i}(\boldsymbol{z}_{t-1}, \boldsymbol{x}_{t:T}))^2 \frac{1}{\sigma^2_{\theta_{\boldsymbol{z}},i}(\boldsymbol{z}_{t-1})} - k \quad + \\ &\sum_i \log\frac{\sigma^2_{\theta_{\boldsymbol{z}},i}(\boldsymbol{z}_{t-1})}{\sigma^2_{\phi,i}(\boldsymbol{z}_{t-1}, \boldsymbol{x}_{t:T})}\Bigg)\end{aligned}$$

Summing over all time steps $t = 1, \ldots, T$:

$$\sum_{t=1}^{T} \mathrm{KL}\big(q_\phi(\boldsymbol{z}_t|\boldsymbol{z}_{t-1}, \boldsymbol{x}_{t:T}) \parallel p_\theta(\boldsymbol{z}_t|\boldsymbol{z}_{t-1})\big) =$$

$$\frac{1}{2}\sum_{t=1}^{T}\left(\sum_i \left(\frac{\sigma^2_{\phi,i}(\boldsymbol{z}_{t-1}, \boldsymbol{x}_{t:T})}{\sigma^2_{\theta_{\boldsymbol{z}},i}(\boldsymbol{z}_{t-1})} + \frac{(\mu_{\theta_{\boldsymbol{z}},i}(\boldsymbol{z}_{t-1}) - \mu_{\phi,i}(\boldsymbol{z}_{t-1}, \boldsymbol{x}_{t:T}))^2}{\sigma^2_{\theta_{\boldsymbol{z}},i}(\boldsymbol{z}_{t-1})} - 1 + \log \frac{\sigma^2_{\theta_{\boldsymbol{z}},i}(\boldsymbol{z}_{t-1})}{\sigma^2_{\phi,i}(\boldsymbol{z}_{t-1}, \boldsymbol{x}_{t:T})}\right)\right) \tag{4}$$

In the estimator (4.7), each term of this sum is evaluated at the sampled previous state $\dot{\boldsymbol{z}}_{t-1}$, both in the parameters of q_ϕ and in those of $p_{\theta_{\boldsymbol{z}}}$. The term -1 inside the sum contributes the constant $-\frac{Td_z}{2}$, where d_z is the dimension of $\boldsymbol{z}_t$. It does not depend on the parameters, which is why it can be dropped from the loss (Section 4.8.1).

E Combiner in the Deep Kalman Filter (DKF)

The **Combiner** is a key component in the DKF model, responsible for integrating information from different sources to produce a unified representation. Specifically, it combines the previous latent state ($\boldsymbol{z}_{t-1}$) with the hidden states of one or more Recurrent Neural Networks (RNNs) to create an output that is fed into the encoder. This output is then used to compute the parameters of the latent distribution ($\boldsymbol{z}_t$).

Purpose of the Combiner

The Combiner's primary role is to:

1. **Integrate Information**: Combine the previous latent state ($\boldsymbol{z}_{t-1}$) with the hidden states of RNNs (e.g., backward RNN, forward RNN) to create a richer representation.
2. **Produce a Unified Output**: Generate a single tensor that can be fed into the encoder to compute the mean and log variance of the latent state ($\boldsymbol{z}_t$).
3. **Flexibility**: Allow for different ways of combining information, such as linear combination or using a Multi-Layer Perceptron (MLP).

Key Concepts

- **Latent State ($\boldsymbol{z}_{t-1}$)**:
  - The latent state from the previous time step, which represents the hidden variables of the model. At $t = 1$, $\boldsymbol{z}_0$ is either a fix tensor of zeros or a learnable parameter initialiazed with zeros.
- **RNN Hidden States ($\boldsymbol{h}_t^i$)**:
  - The output of one or more RNNs (e.g., backward RNN, forward RNN) that process the input sequence. These hidden states capture temporal dependencies in the data.
- **Combination Strategy**:
  - The method used to combine $\boldsymbol{z}_{t-1}$ with the RNN hidden states. This can be a simple linear combination or a more complex MLP-based combination.

Combiner Types

The Combiner is implemented in two main ways:

1. **Linear Combiner**: Combines $\boldsymbol{z}_{t-1}$ and RNN hidden states using a weighted sum (linear combination).
2. **MLP Combiner**: Combines $\boldsymbol{z}_{t-1}$ and RNN hidden states using a Multi-Layer Perceptron (MLP).

E.1 Linear Combiner

The **Linear Combiner** performs a weighted sum of the previous latent state ($\boldsymbol{z}_{t-1}$) and the RNN hidden states. The weights are controlled by a parameter α_i, which determines the contribution of each input.

Key Parameters

- z_dim: Dimension of the latent state ($\boldsymbol{z}_{t-1}$).
- rnn_dims: Dimensions of the RNN hidden states. This can be a single dimension or a list of dimensions if multiple RNNs are used.
- $output_dim$: Dimension of the output tensor.
- α^i: Weight for the i-th RNN hidden state. This determines how much influence the mapped $\boldsymbol{z}_{t-1}$ has on the combination.

Formula

$$\boldsymbol{combined}_t^i = \alpha^i \cdot \text{mapped}(\boldsymbol{z}_{t-1}) + (1 - \alpha^i) \cdot \boldsymbol{h}_t^i$$

Where:

- $\text{mapped}(\boldsymbol{z}_{t-1}) = \tanh(\mathbf{W}^i \boldsymbol{z}_{t-1} + \mathbf{c}^i)$: the previous latent state mapped to the dimension of the i-th RNN hidden state by a linear layer followed by the activation function (tanh in all experiments).
- $\boldsymbol{h}_t^i$: The hidden state output of the i-th RNN at time t.

With a single RNN and $\alpha = 0.5$, this is the combiner of Krishnan et al.'s follow-up work [4], $\frac{1}{2}\big(\tanh(\mathbf{W}\boldsymbol{z}_{t-1} + \mathbf{c}) + \boldsymbol{h}_t\big)$, followed by a linear layer.

Two sets of weights are kept: `alpha_init`, meant for the first time step, where $\boldsymbol{z}_0$ is a fixed or learned constant that carries no information about the sequence, and `alpha`, used for the following steps.

How It Works

1. **Mapping $\boldsymbol{z}_{t-1}$**:
   - The previous latent state ($\boldsymbol{z}_{t-1}$) is mapped to the dimension of each RNN hidden state by separate linear layers, each followed by the activation function.
2. **Weighted Sum**:
   - The mapped $\boldsymbol{z}_{t-1}$ and the RNN hidden state are combined using a weighted sum. The weight α^i controls the contribution of the mapped $\boldsymbol{z}_{t-1}$, while $(1 - \alpha^i)$ controls the contribution of the RNN hidden state.
3. **Concatenation**:
   - The combined tensors for all RNN hidden states are concatenated along the feature dimension.
4. **Final Linear Layer**:
   - The concatenated tensor is passed through a final linear layer to produce the output tensor of dimension `output_dim`.

Example

Suppose:

- $\alpha^i = 0.3$
- $\text{mapped}(\boldsymbol{z}_{t-1}) = \boldsymbol{mz}$
- $\boldsymbol{h}_t^i = \boldsymbol{rnn_h}$

The formula becomes:

$$\begin{aligned}\boldsymbol{combined}^i &= 0.3 \cdot \boldsymbol{mz} + (1 - 0.3) \cdot \boldsymbol{rnn_h} \\ &= 0.3 \cdot \boldsymbol{mz} + 0.7 \cdot \boldsymbol{rnn_h}\end{aligned}$$

This means:

- 30% of the contribution comes from the mapped latent state mapped.
- 70% of the contribution comes from the RNN hidden state `rnn_h`.

E.2 MLP Combiner

The **MLP Combiner** uses a Multi-Layer Perceptron (MLP) to combine the previous latent state ($\boldsymbol{z}_{t-1}$) and the RNN hidden states. This allows for more complex interactions between the inputs.

Key Parameters

- `z_dim`: Dimension of the latent state ($\boldsymbol{z}_{t-1}$).
- `rnn_dims`: Dimensions of the RNN hidden states.
- `output_dim`: Dimension of the output tensor.
- `layers_dim`: List of dimensions for the hidden layers in the MLP.
- `activation`: Activation function (e.g., tanh or ReLU).

How It Works

1. **Concatenation**:
   - The previous latent state ($\boldsymbol{z}_{t-1}$) and all RNN hidden states are concatenated along the feature dimension.
2. **MLP Layers**:
   - The concatenated tensor is passed through a series of linear layers with activation functions in between.
   - The first layer maps the concatenated input to the first hidden layer dimension.
   - Intermediate layers map the output of one hidden layer to the next.
   - The final layer maps the last hidden layer to the output dimension (`output_dim`).
3. **Output**:
   - The output of the MLP is the combined tensor, which is then fed into the encoder.

Example

Suppose:

- $\boldsymbol{z}_{t-1}$ has a dimension of 16.
- Two RNN hidden states: h_t^1 (dimension 32) and h_t^2 (dimension 64).

The concatenated input will have a dimension of $16 + 32 + 64 = 112$. This concatenated tensor is passed through an MLP.

E.3 Combiner Factory

The **Combiner Factory** is a utility class that creates the appropriate Combiner based on the specified type (linear or mlp). This allows for flexibility in choosing the combination strategy.

Key Parameters

- `combiner_type`: Type of Combiner to use (linear or mlp).
- **Other Parameters**: Depend on the type of Combiner (e.g., α^i, `layers_dim`).

How It Works

- If `combiner_type` = `linear`, the factory initializes a `DKF_CombinerLinear` with the specified parameters.
- If `combiner_type` = `mlp`, the factory initializes a `DKF_CombinerMLP` with the specified parameters.

E.4 Summary of the Combiner's Role in the DKF

The Combiner is responsible for integrating the previous latent state ($\boldsymbol{z}_{t-1}$) with the hidden states of RNNs to produce a unified representation. This representation is then used by the encoder to compute the parameters of the latent distribution ($\boldsymbol{z}_t$). The Combiner can be either a simple linear combination or a more complex MLP-based combination, depending on the model's requirements.
By using the Combiner, the DKF model can effectively leverage both the temporal dependencies captured by RNNs and the latent state dynamics to model complex time series data.

F Results of Deep Kalman Filter Models

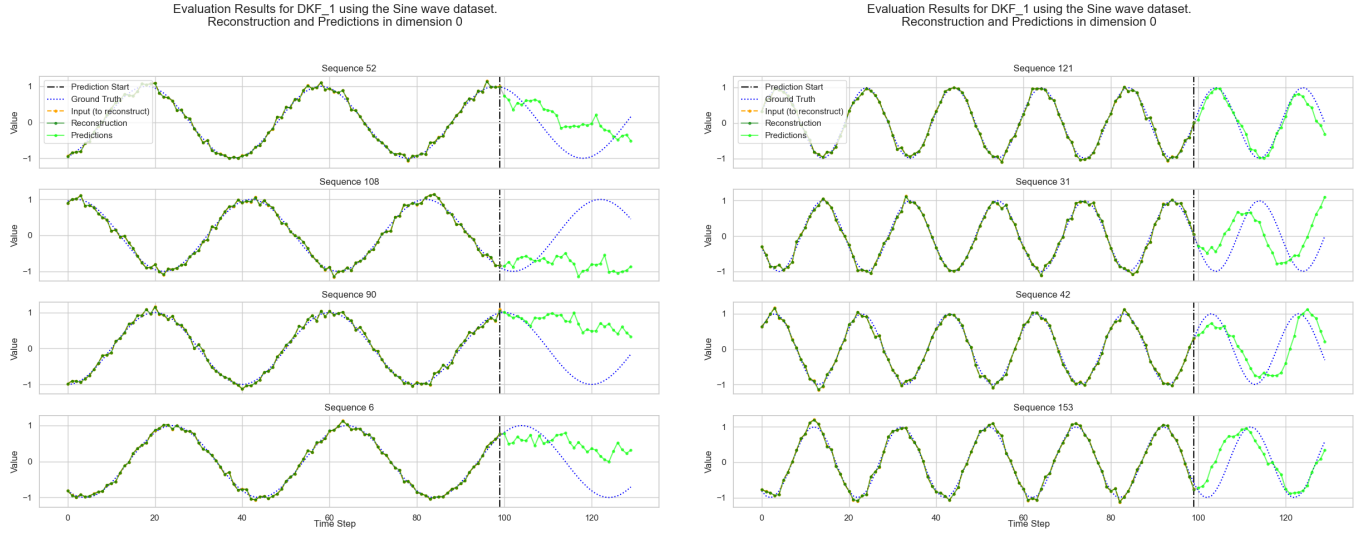
This appendix presents the results of the vanilla DKF and of the AR DKF, FAR DKF and BAR DKF on two kinds of data: simple sine waves, and windows of a trajectory of the Lorenz attractor. Gaussian noise is added to the observed part of each sequence. The model reconstructs that part and then forecasts the following steps.

How to read these results

- **Errors are measured against the clean signal.** The reconstruction and prediction MSEs compare the model's output with the ground truth *before* noise was added. The "MSE of the noise" is the error one would make by simply returning the noisy input.
- **Reconstruction at the noise level means no denoising.** For all the models, the reconstruction MSE is close to the MSE of the noise. In the figures, the reconstruction follows the noisy input point by point rather than the clean curve. The inference network passes each observation, noise included, through the latent state, and the decoder reproduces it. A good reconstruction MSE is therefore *not* evidence that the model has learned the dynamics; the forecasting error is the informative quantity.
- **Forecasts are single sampled paths.** `predict` samples one latent trajectory and one path of the dynamics. Its error includes sampling noise and would be lower for the mean of the predictive distribution.
- **Repeated runs.** Each Lorenz experiment was trained four times. The figures of each dataset show the first run; the summary at the end of this appendix gives the mean and the spread over the four runs. The sine-wave experiments were trained once.

DKF with Simple Sine Waves

The vanilla DKF was tested on two sine-wave datasets with different frequencies. It follows the observed part of the sequences closely, but its forecasts lose track of the signal. With $f = 1$ they flatten out and lose the amplitude; with $f = 2$ they keep oscillating at roughly the right frequency but drift out of phase within one or two periods. The transition model alone has not learned the periodic dynamics.



(a) DKF on sine wave with $f = 1$.

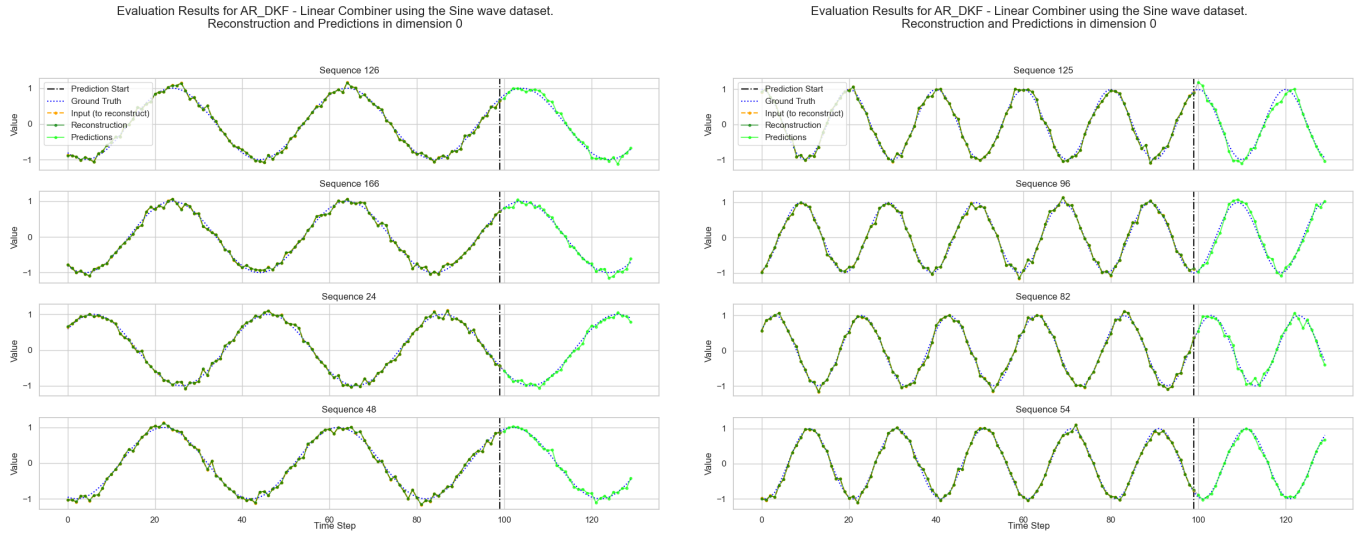
(b) DKF on sine wave with $f = 2$.

Figure 2: DKF reconstruction and prediction on two sine-wave datasets.

AR DKF with Simple Sine Waves

The AR DKF was tested on the same sine wave datasets as for the DKF. The AR DKF forecasts both sine waves far better than the vanilla DKF: the forecasts keep the phase, the frequency and the amplitude of

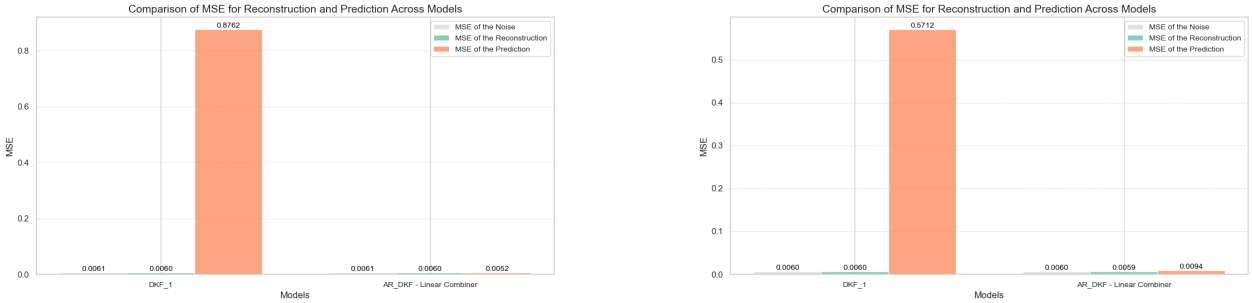
the signal over the whole prediction window. The prediction MSE is 0.876 for the DKF and 0.0052 for the AR DKF with $f = 1$, and 0.571 and 0.0094 with $f = 2$ (Figure 4). For a sine wave of amplitude 1, always predicting 0 would give an MSE of 0.5: on both datasets the DKF forecasts are worse than this trivial predictor, while the AR DKF forecasts have an error of the same order as the noise of the observations. The difference is large enough to be conclusive even with a single run. It supports the idea behind the AR DKF: giving the transition model direct access to the history lets it carry the information needed for forecasting.



(a) AR DKF on sine wave with $f = 1$.

(b) AR DKF on sine wave with $f = 2$.

Figure 3: Examples of AR DKF Reconstruction and Prediction on two simple sine wave datasets.



(a) Sine wave with $f = 1$.

(b) Sine wave with $f = 2$.

Figure 4: Reconstruction and prediction MSE of the DKF and the AR DKF on the two sine-wave datasets.

Comparison of AR DKF, FAR DKF, and BAR DKF on Lorenz Attractor

The AR DKF, FAR DKF, and BAR DKF were compared using three sets of data from the Lorenz attractor. The datasets were designed to test the models' ability to generalize under varying degrees of dissimilarity between training and testing data. Each experiment was repeated four times with the same settings; the figures below show the first run.

Configurations

All the subsequent results are obtained with configurations given in the tables below.

Table 1: Args Configuration

Parameter	Value
dataset	lorenz
lorenz_initial_point	2, 1, 1
lorenz_points	20000
lorenz_range	Dataset 1: [4500, 6500] Dataset 2: [8600, 10500] Dataset 3: [11000, 13000]
loss_function	weighted_mse
noise_var	0.006
normalization_type	standardize
train_seq_len	100
test_seq_len (prediction)	100

Table 2: Training Configuration

Parameter	Value
learning_rate	0.001
num_epochs	120
pred_batch_size	32
train_batch_size	32
val_batch_size	16

Table 3: Models Configuration

Parameter \ Model	AR DKF	FAR DKF	BAR DKF
activation	tanh	tanh	tanh
combiner_alpha	0.5	0.5, 0.5	0.5
combiner_alpha_init	0.1	0.1, 0.1	0.1
combiner_layers_dim	64	64	64
combiner_learnable_alpha	Yes	Yes, Yes	Yes
combiner_output_dim	64	64	64
combiner_type	linear	linear	linear
decoder_layers_dim	64, 32	64, 32	64, 32
device	cpu	cpu	cpu
dropout_p	0.0	0.0	0.0
encoder_layers_dim	64	64	64
latent_transition_layers_dim	64	64	64
learnable_init_x	No	No	No
learnable_init_z	Yes	Yes	Yes
pz_f_rnn_hidden_size	64	64	64
pz_f_rnn_num_layers	1	1	1
q_b_rnn_hidden_size	64	64	64
q_b_rnn_num_layers	1	1	1
x_dim	3	3	3
z_dim	16	16	16
q_f_rnn_hidden_size		64	
q_f_rnn_num_layers		1	

Dataset 1

The dataset (*before the addition of Gaussian noise*) is shown in the figure 5 below.

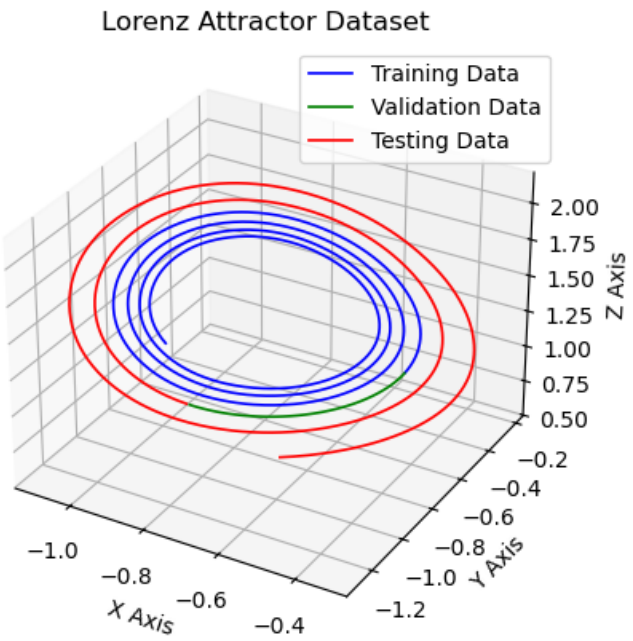
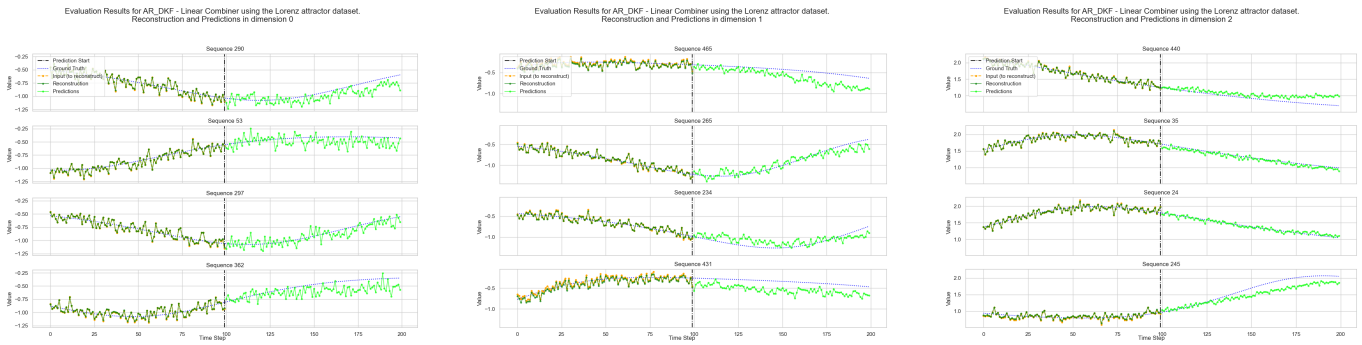


Figure 5: Lorenz attractor - Range [4500, 6500]

Figure 6 below shows the results of the AR DKF model for four sequences from the dataset in each dimension.



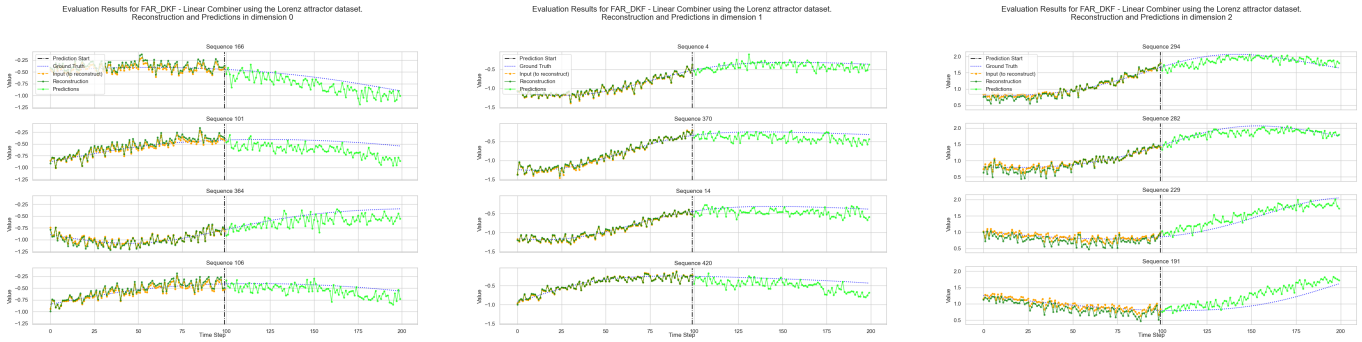
(a) Reconstruction and Prediction in the 1st dimension.

(b) Reconstruction and Prediction in the 2nd dimension.

(c) Reconstruction and Prediction in the 3rd dimension.

Figure 6: Examples of Reconstruction and Prediction for the AR DKF model.

Figure 7 below shows the results of the FAR DKF model on the same sequences.
And Figure 8 shows the results of the BAR DKF.

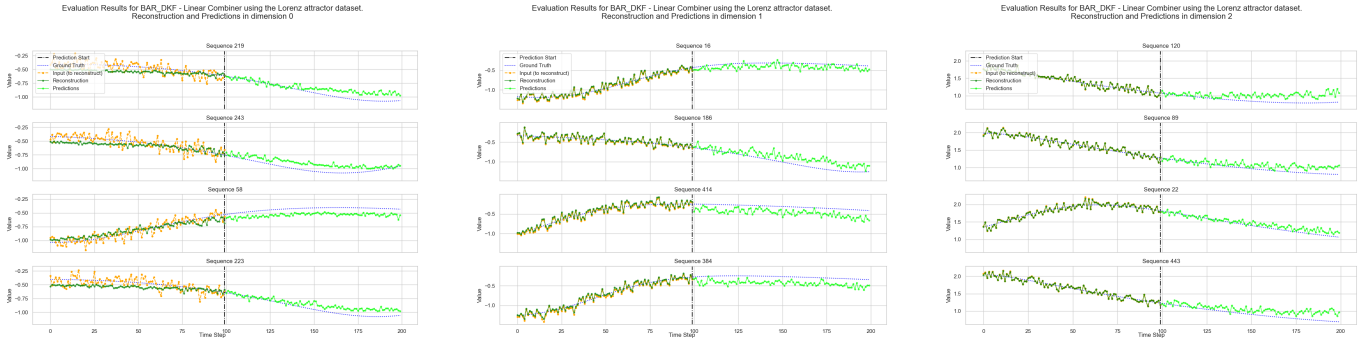


(a) Reconstruction and Prediction in the 1st dimension.

(b) Reconstruction and Prediction in the 2nd dimension.

(c) Reconstruction and Prediction in the 3rd dimension.

Figure 7: Examples of Reconstruction and Prediction for the FAR DKF model.



(a) Reconstruction and Prediction in the 1st dimension.

(b) Reconstruction and Prediction in the 2nd dimension.

(c) Reconstruction and Prediction in the 3rd dimension.

Figure 8: Examples of Reconstruction and Prediction for the BAR DKF model.

Figure 9 below shows a comparison of the Mean Square Error for the reconstruction and the prediction tasks for each model. The MSE of the noise is plotted for reference.
The three models have prediction errors of the same order: 0.0269 for the AR DKF, 0.0309 for the FAR DKF and 0.0218 for the BAR DKF. The AR DKF and the BAR DKF reconstruct at the noise level. The FAR DKF reconstruction error (0.0092) is above it, i.e. worse than simply returning the noisy input.

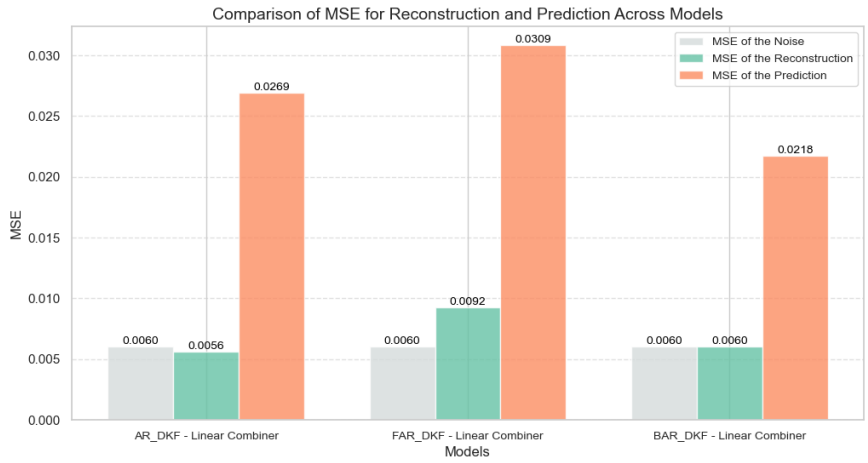


Figure 9: Reconstruction and Prediction MSE Comparison for AR DKF, FAR DKF, and BAR DKF on dataset 1.

Figure 10 below shows a comparison of the Mean Square Error for each predicted point for each model. As expected, the error grows with the forecasting horizon, since errors accumulate along the sampled path.

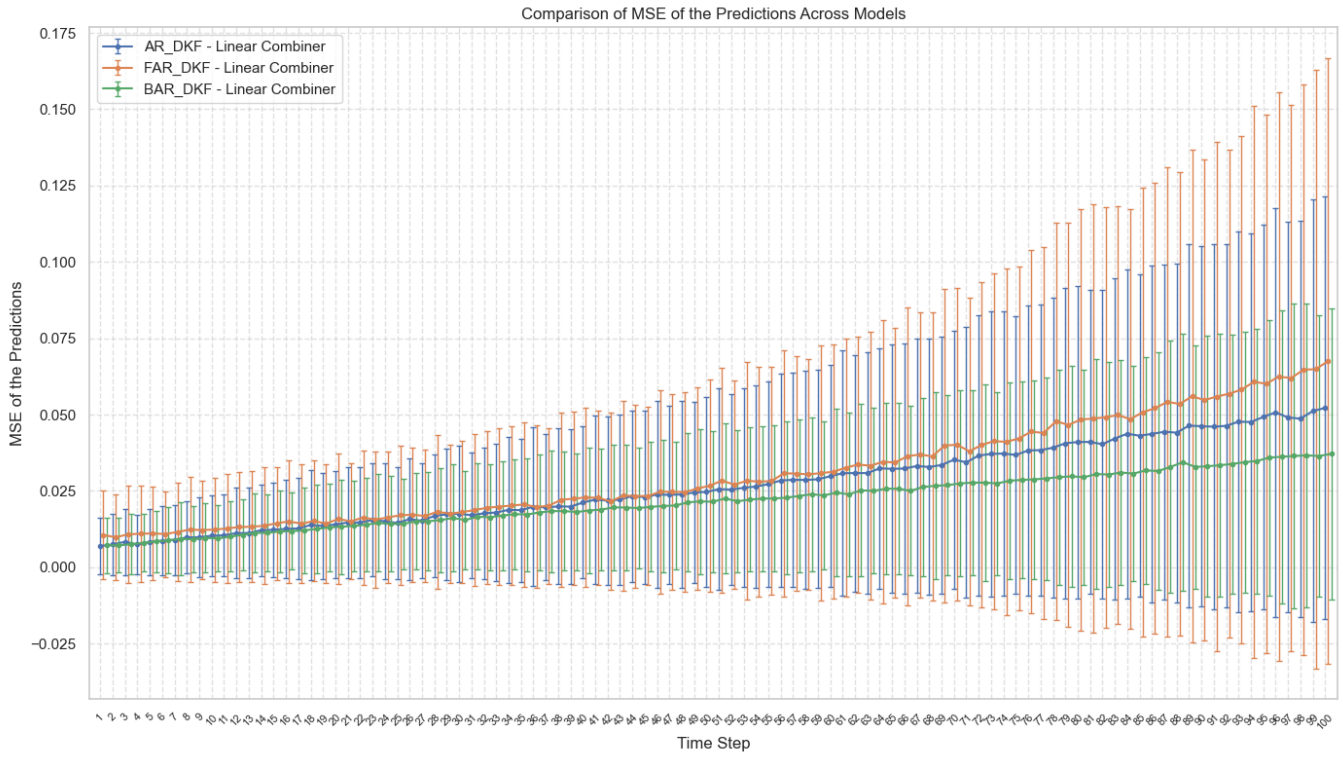


Figure 10: MSE Comparison for AR DKF, FAR DKF, and BAR DKF for each prediction points of test sequences of dataset 1.

The same comparisons were done on the dataset 2 and 3. The results are shown below.

Dataset 2

The dataset (*before the addition of Gaussian noise*) is shown in figure 11.

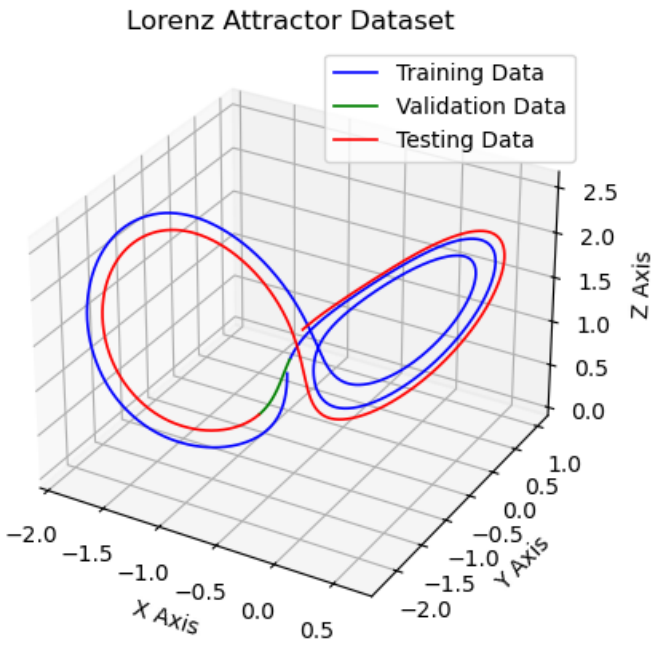
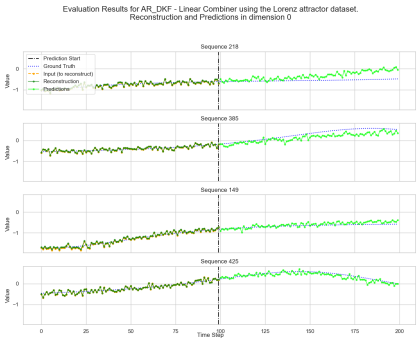
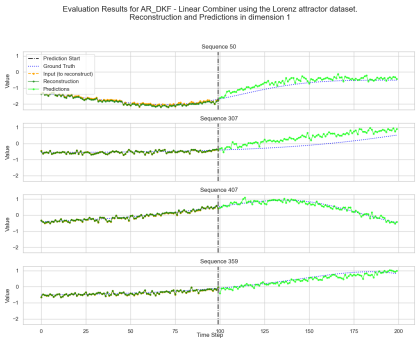


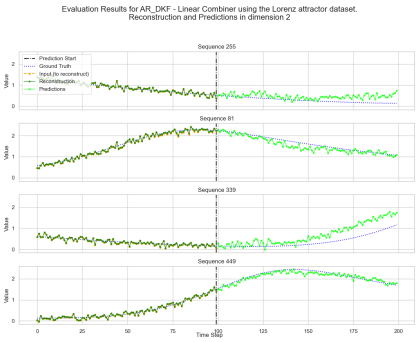
Figure 11: Lorenz attractor - Range [8600, 10500]



(a) Reconstruction and Prediction in the 1st dimension.

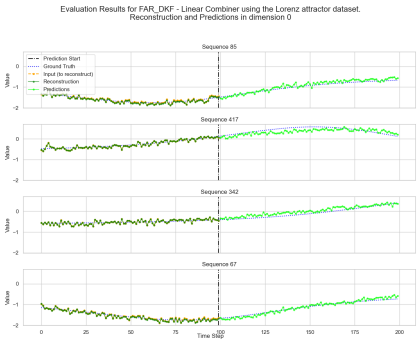


(b) Reconstruction and Prediction in the 2nd dimension.

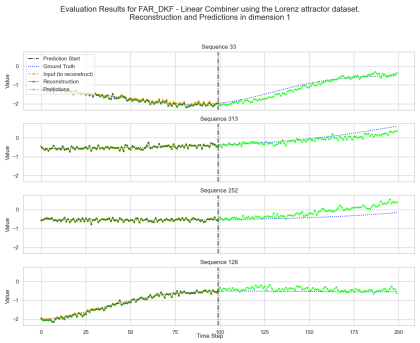


(c) Reconstruction and Prediction in the 3rd dimension.

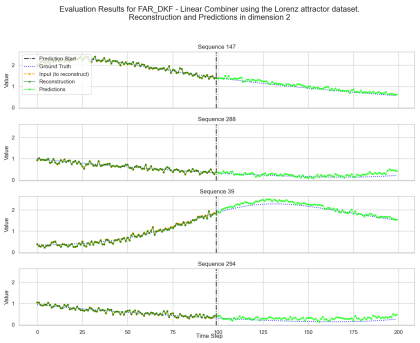
Figure 12: Examples of Reconstruction and Prediction for the AR DKF model.



(a) Reconstruction and Prediction in the 1st dimension.

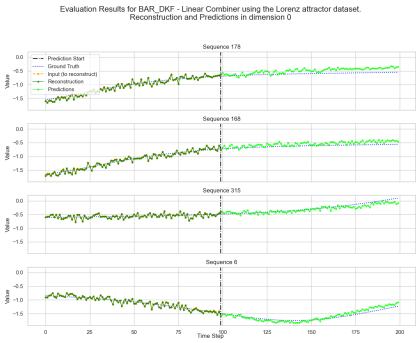


(b) Reconstruction and Prediction in the 2nd dimension.

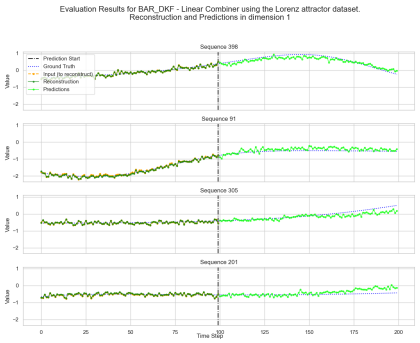


(c) Reconstruction and Prediction in the 3rd dimension.

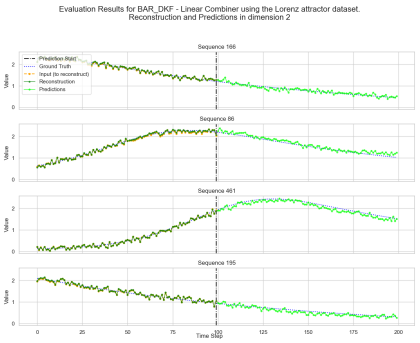
Figure 13: Examples of Reconstruction and Prediction for the FAR DKF model.



(a) Reconstruction and Prediction in the 1st dimension.



(b) Reconstruction and Prediction in the 2nd dimension.



(c) Reconstruction and Prediction in the 3rd dimension.

Figure 14: Examples of Reconstruction and Prediction for the BAR DKF model.

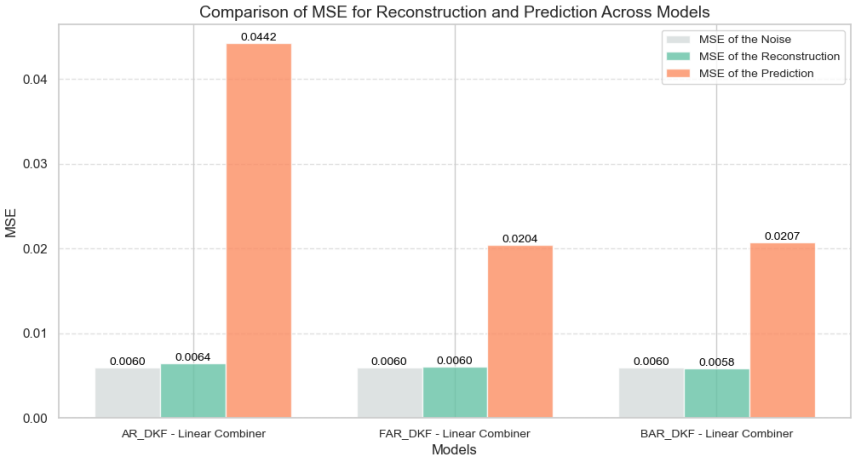


Figure 15: Reconstruction and Prediction MSE Comparison for AR DKF, FAR DKF, and BAR DKF on dataset 2.

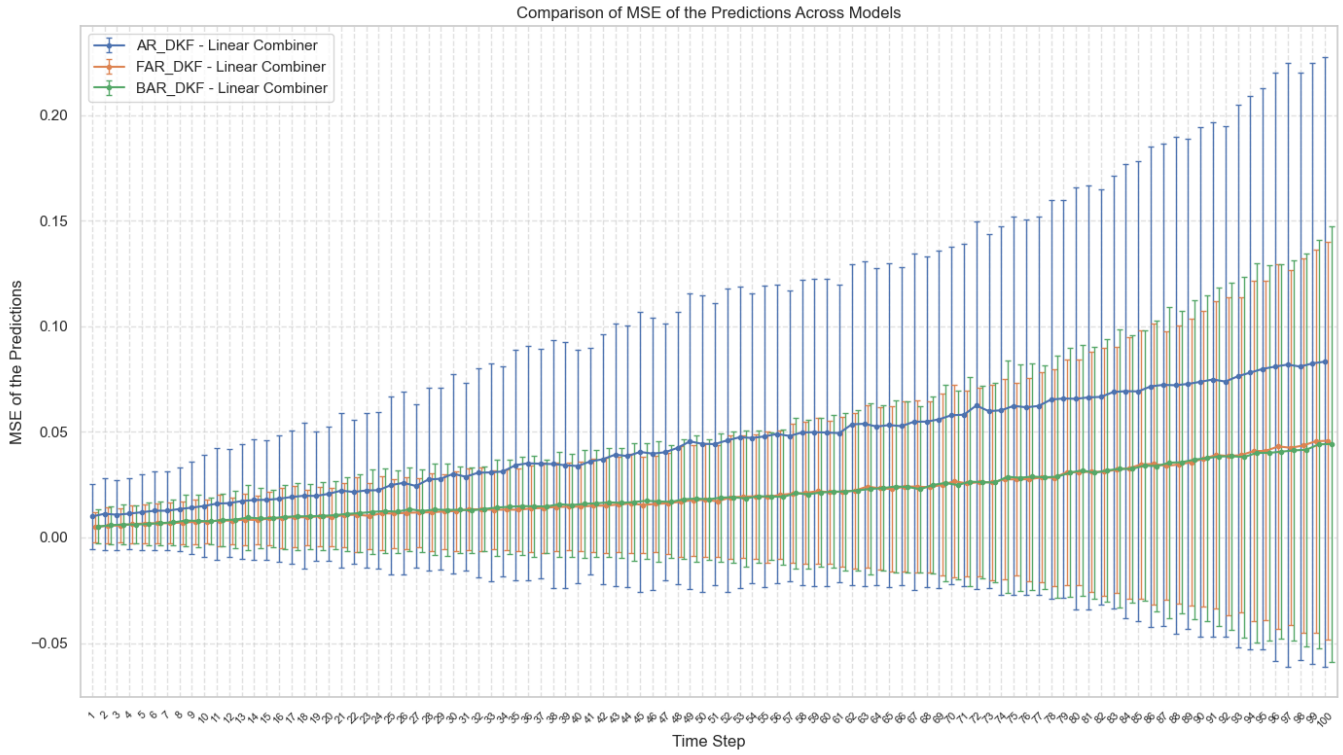


Figure 16: MSE Comparison for AR DKF, FAR DKF, and BAR DKF for each prediction points of test sequences of dataset 2.

Dataset 3

The dataset (*before the addition of Gaussian noise*) is shown in figure 17 below.

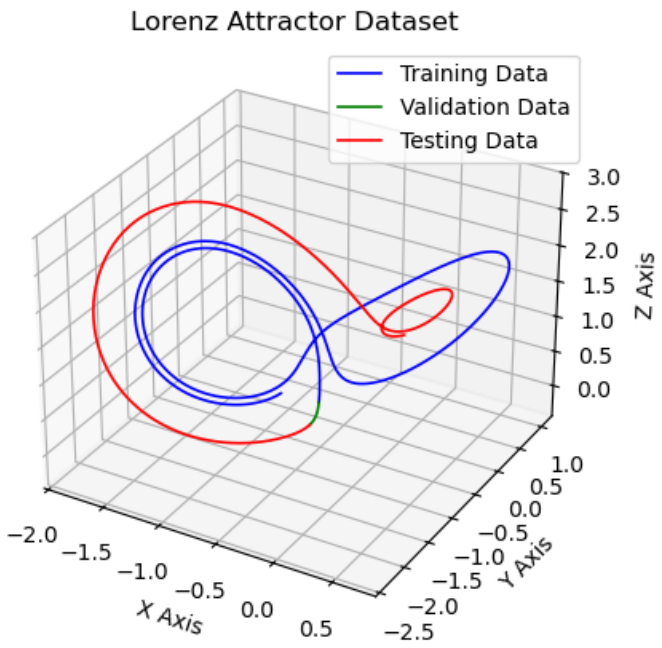
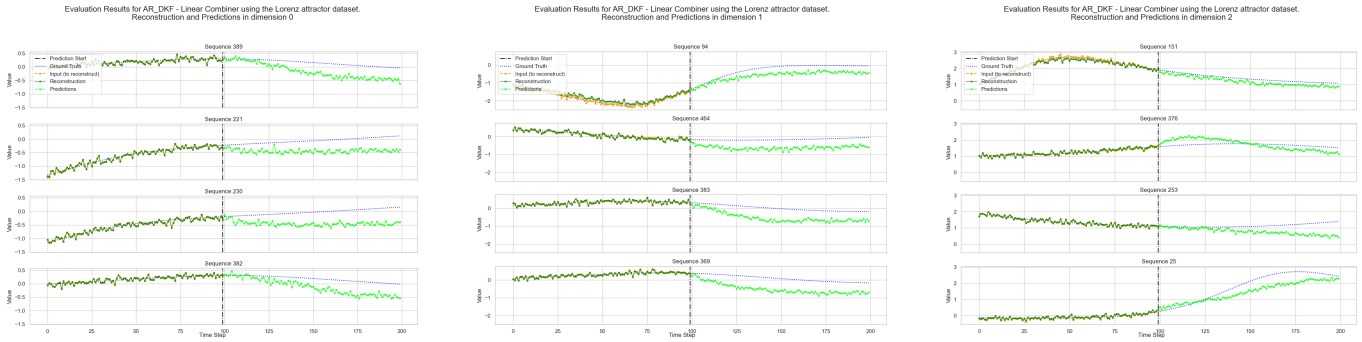


Figure 17: Lorenz attractor - Range [11000, 13000]

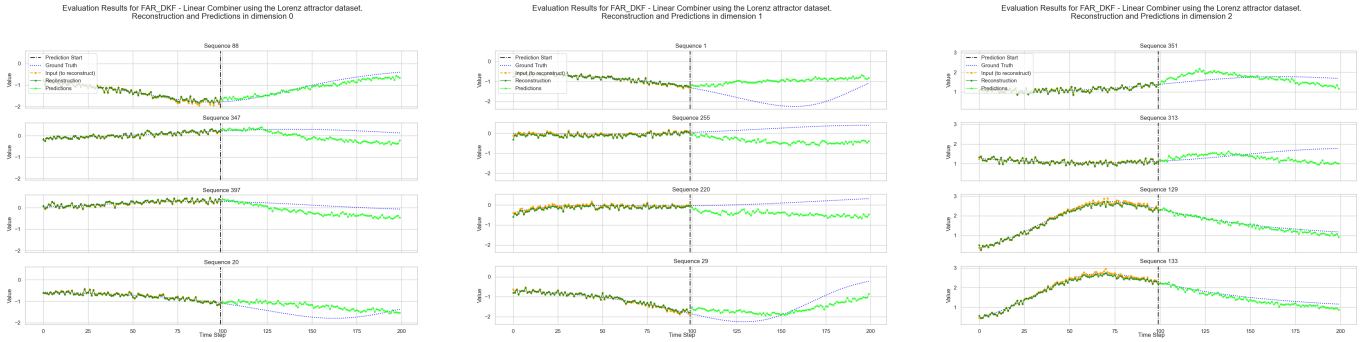


(a) Reconstruction and Prediction in the 1st dimension.

(b) Reconstruction and Prediction in the 2nd dimension.

(c) Reconstruction and Prediction in the 3rd dimension.

Figure 18: Examples of Reconstruction and Prediction for the AR DKF model.

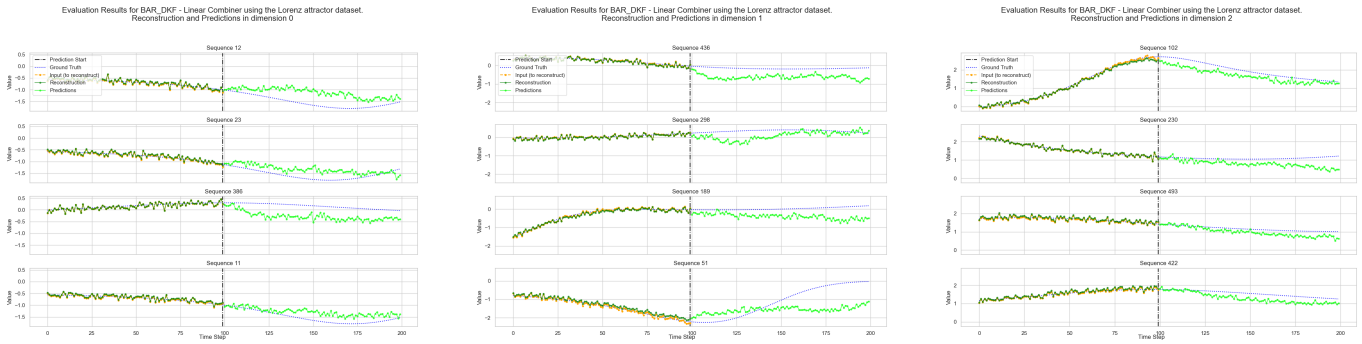


(a) Reconstruction and Prediction in the 1st dimension.

(b) Reconstruction and Prediction in the 2nd dimension.

(c) Reconstruction and Prediction in the 3rd dimension.

Figure 19: Examples of Reconstruction and Prediction for the FAR DKF model.



(a) Reconstruction and Prediction in the 1st dimension.

(b) Reconstruction and Prediction in the 2nd dimension.

(c) Reconstruction and Prediction in the 3rd dimension.

Figure 20: Examples of Reconstruction and Prediction for the BAR DKF model.

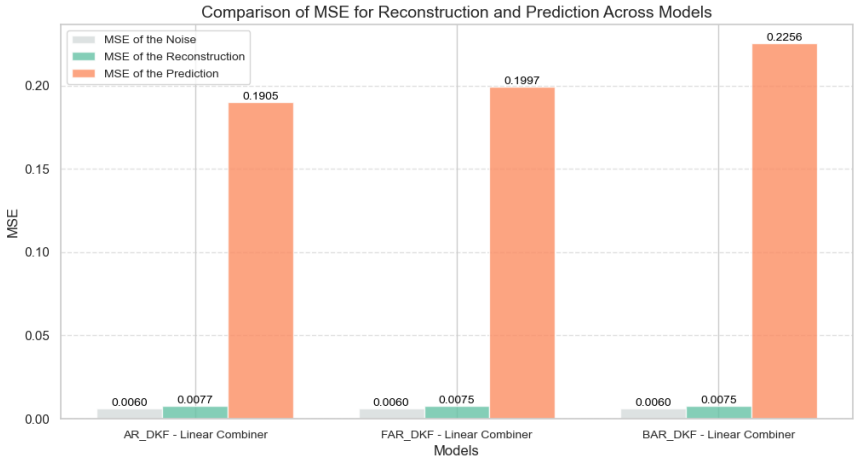


Figure 21: Reconstruction and Prediction MSE Comparison for AR DKF, FAR DKF, and BAR DKF on dataset 3.

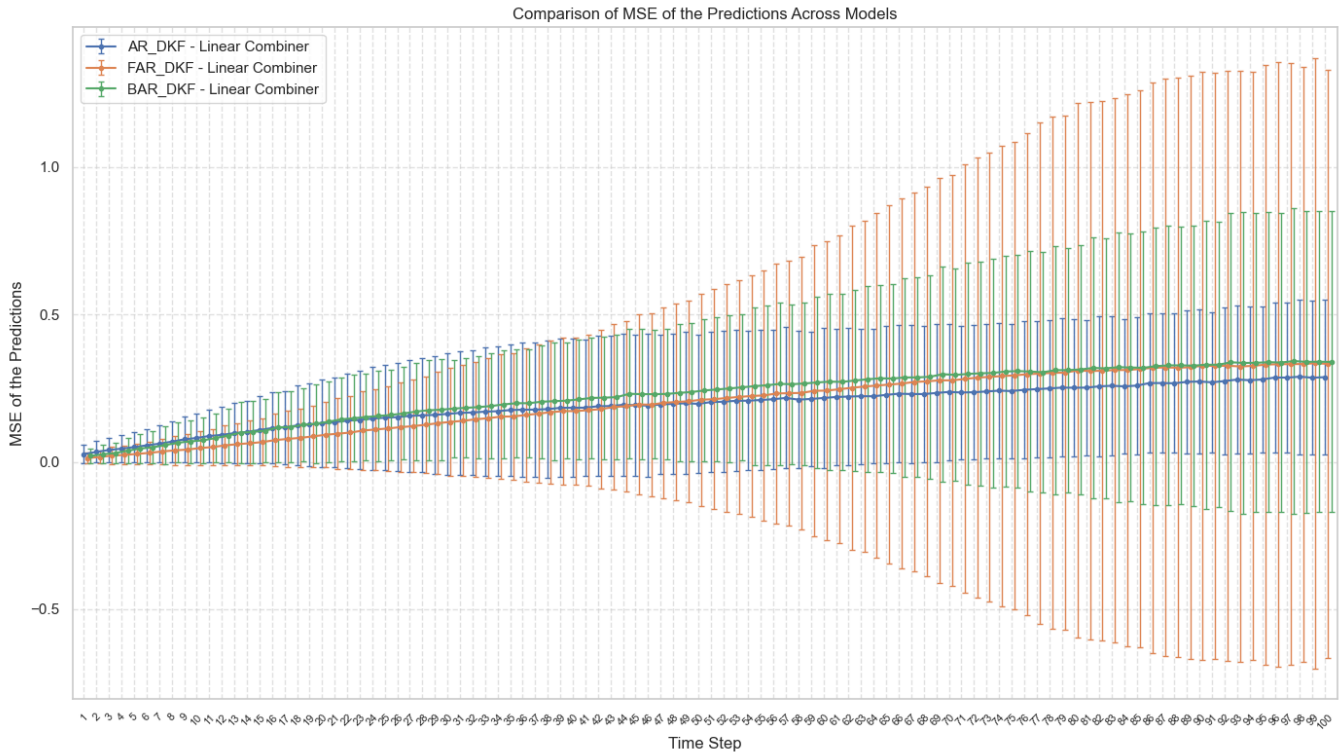


Figure 22: MSE Comparison for AR DKF, FAR DKF, and BAR DKF for each prediction points of test sequences of dataset 3.

Summary of the Lorenz experiments

The code does not fix a random seed, so each training run starts from a different initialisation and sees the minibatches in a different order. Table 4 gives the forecasting and reconstruction errors over the four runs of each experiment.

Dataset	Model	Prediction MSE: mean (std. dev.)	Runs with the lowest error
1	AR DKF	0.0357 (0.0202)	2 of 4
	FAR DKF	0.0315 (0.0063)	1 of 4
	BAR DKF	0.0280 (0.0044)	1 of 4
2	AR DKF	0.0244 (0.0135)	3 of 4
	FAR DKF	0.0361 (0.0232)	1 of 4
	BAR DKF	0.0621 (0.0695)	0 of 4
3	AR DKF	0.1974 (0.0283)	1 of 4
	FAR DKF	0.1418 (0.0513)	2 of 4
	BAR DKF	0.1711 (0.0597)	1 of 4

Table 4: Lorenz experiments: prediction MSE over four training runs. The MSE of the noise is 0.0060.

- **Reconstruction.** In almost every run, all three models reconstruct close to the noise level (mean reconstruction MSE between 0.0050 and 0.0071, except 0.0119 for the AR DKF on dataset 3, due to one run at 0.0225). They reproduce the noisy input rather than denoise it.
- **Prediction.** The spread between runs of the *same* model is as large as the differences between models, and no model is consistently best: the AR DKF has the lowest error in three runs out of four on dataset 2, the FAR DKF the lowest mean error on dataset 3, and on dataset 1 the three means are within one standard deviation of each other. The BAR DKF has one poor run on dataset 2 (0.166), which drives its mean and spread there. Dataset 3 remains the hardest for all models.

Evaluation of the ELBO on test data

The theory of Section 4.10.3 makes a prediction about the ELBO, not about forecasting: the FAR DKF and the BAR DKF have inference networks with the structure of the true posterior (Proposition 2); the AR DKF does not. Their gap $\mathrm{KL}\big(q_\phi \,\|\, p_\theta(\mathbf{z}_{1:T} \mid \mathbf{x}_{1:T})\big)$ between the ELBO and the log-likelihood should therefore be smaller.

To test this, the script `evaluate_elbo.py` evaluates each trained model on the noisy test sequences (the same noise for all models) with two quantities, reported per time step:

- the ELBO, estimated as during training and averaged over 20 trajectories per sequence;
- the importance-weighted bound [5]

$$\mathcal{L}_K = \mathbb{E}\left[\log \frac{1}{K}\sum_{k=1}^{K} \frac{p_\theta(\mathbf{x}_{1:T}, \mathbf{z}_{1:T}^{(k)})}{q_\phi(\mathbf{z}_{1:T}^{(k)} \mid \mathbf{x}_{1:T})}\right], \qquad \mathbf{z}_{1:T}^{(k)} \sim q_\phi(\mathbf{z}_{1:T} \mid \mathbf{x}_{1:T}),$$

  for $K = 1, 10, 100$. It satisfies $\mathcal{L}_1 = \mathcal{L}$ (the ELBO) and $\mathcal{L}_K \le \log p_\theta(\mathbf{x}_{1:T})$, and it increases with K. Hence $\mathcal{L}_{100} - \mathcal{L}$ is a *lower bound* on the gap.

As a check of the implementation, $\mathcal{L}_1$ agrees with the ELBO within its confidence interval in every evaluation, and $\mathcal{L}_K$ increases with K. The ELBO was evaluated after each of the last three runs of every dataset (and after the first run for dataset 3).

- **The ELBO varies more between runs than between models.** Its standard deviation over runs of the same model is mostly between 5 and 13 nats per time step, while the differences between the means of the three models are of the same size or smaller, and the model with the best ELBO changes from one run to the next. The AR DKF has the lowest mean ELBO on dataset 2 (the lowest in two runs out of three), yet the lowest forecasting error there: the ELBO and the forecasting error measure different things.

Dataset	Model	ELBO: mean (std. dev.)	Gap $\mathcal{L}_{100} - \mathcal{L}$: mean [min, max]
1	AR DKF	-13.8 (7.5)	0.62 [0.58, 0.69]
	FAR DKF	-23.4 (8.6)	0.65 [0.60, 0.68]
	BAR DKF	-17.2 (13.1)	0.63 [0.56, 0.73]
2	AR DKF	-30.8 (0.8)	0.69 [0.66, 0.70]
	FAR DKF	-22.9 (4.7)	0.64 [0.62, 0.67]
	BAR DKF	-25.4 (12.5)	0.63 [0.53, 0.69]
3	AR DKF	-41.3 (8.5)	0.80 [0.72, 0.95]
	FAR DKF	-38.8 (7.8)	0.81 [0.72, 0.95]
	BAR DKF	-37.5 (7.4)	0.75 [0.73, 0.79]

Table 5: Lorenz experiments: ELBO and estimated ELBO gap on the test sequences (in nats per time step; higher ELBO is better). ELBO over three runs (four for dataset 3); gap over three runs.

- **The estimated gap is the same for the three models**, at about 0.6–0.8 nats per time step. With $K = 100$, the importance-weighted bound cannot detect the difference that the theory predicts. The bound is still rising between $K = 10$ and $K = 100$ (by 0.2 to 0.4 nats per time step), so it has not converged and the true gaps are larger than these estimates. Comparing them would require much larger K or a better proposal distribution.

In short, the experiments do not show the advantage of the FAR DKF and the BAR DKF that the d-separation argument suggests. What they show clearly is that, at this scale, the variability between training runs is larger than the differences between the three inference networks.

ANNEX

I Bayesian Networks: Paths and D-Separation

This is a short overview. A complete treatment is given by Koller and Friedman [6].

Bayesian networks

A Bayesian network is a directed acyclic graph (DAG) whose nodes are random variables, together with the factorisation

$$p(v_1, \dots, v_n) = \prod_{k=1}^{n} p(v_k \mid \mathrm{pa}(v_k)),$$

where $\mathrm{pa}(v_k)$ are the parents of v_k. The graph encodes conditional independences, which d-separation lets us read directly from it.

Paths and the three elementary connections

A *path* between two nodes is a sequence of adjacent nodes, **regardless of the direction of the edges**. For example, in $A \leftarrow B \rightarrow C$ there is a path from A to C even though no arrow goes from A to C. Each intermediate node of a path takes one of three roles, depending on the two edges that meet at it.

1. **Chain** $A \rightarrow B \rightarrow C$. A influences C through B.
2. **Fork** (common cause) $A \leftarrow B \rightarrow C$. A and C are correlated because they share the cause B.
3. **Collider** (common effect) $A \rightarrow B \leftarrow C$. A and C are independent causes of B.

Blocked paths

Let $\mathcal{S}$ be the set of observed (conditioned-on) variables. A path is **blocked** by $\mathcal{S}$ if it contains at least one intermediate node B such that

- B is a chain or fork node and $B \in \mathcal{S}$; or
- B is a collider and neither B nor any of its descendants is in $\mathcal{S}$.

Otherwise the path is **active**. Two sets of variables are **d-separated** by $\mathcal{S}$ if every path between them is blocked. If $\mathcal{X}$ and $\mathcal{Y}$ are d-separated by $\mathcal{S}$, then $\mathcal{X} \perp\!\!\!\perp \mathcal{Y} \mid \mathcal{S}$ in every distribution that factorises according to the graph.

The two mechanisms, with examples

Chains and forks: screening off. Observing the middle node of a chain or a fork *blocks* the path.

Chain. Let $A =$ "it rains", $B =$ "the grass is wet", $C =$ "someone slips on the grass": $A \rightarrow B \rightarrow C$. Without information about the grass, knowing that it rains makes slipping more likely. But once we know

whether the grass is wet, the rain tells us nothing more about slipping: all its influence goes through the wetness. Formally, $p(C \mid A, B) = p(C \mid B)$. The observed B *screens off* C from A.

Fork. Let $B =$ "the patient has the flu", $A =$ "fever", $C =$ "cough": $A \leftarrow B \rightarrow C$. Fever and cough are correlated in the population, because both are symptoms of the flu. Among patients known to have the flu, observing a fever does not make a cough more likely: $A \perp\!\!\!\perp C \mid B$.

Colliders: explaining away. A collider works the other way round: the path is blocked as long as the collider is *unobserved*, and observing it (or one of its descendants) *opens* the path.

Let $A =$ "it rained", $C =$ "the sprinkler was on", $B =$ "the grass is wet": $A \rightarrow B \leftarrow C$. Rain and sprinkler are independent: $A \perp\!\!\!\perp C$. But once we observe that the grass is wet, learning that it rained makes the sprinkler *less* likely, because the rain already explains the wet grass. Observing the common effect makes its causes dependent. This phenomenon, and only this one, is called **explaining away**.

In words

Information flows through chains and forks unless the middle node is known. It flows through a collider only if the collider (or something it causes) is known. Two variables are conditionally independent when every route between them is cut by one of these rules.

Application to the DKF

In the graph of Figure 4.1, the latent states form a chain $\mathbf{z}_1 \rightarrow \mathbf{z}_2 \rightarrow \cdots$, each $\mathbf{z}_t$ has a child $\mathbf{x}_t$, and each input $\mathbf{u}_t$ is a parent of $\mathbf{z}_t$.

- Given $\mathbf{z}_{t-1}$, every path from $\mathbf{x}_{t-1}$, $\mathbf{z}_{t-2}$ or $\mathbf{u}_{t-1}$ to $\mathbf{z}_t$ passes through $\mathbf{z}_{t-1}$ as a chain or fork node, and is blocked. This gives the factorisation of the generative model and the structure of the posterior (Sections 4.2.1 and 4.3).
- The future observation $\mathbf{x}_{t+1}$ is *not* d-separated from $\mathbf{z}_t$ by $\mathbf{z}_{t-1}$: the path $\mathbf{z}_t \rightarrow \mathbf{z}_{t+1} \rightarrow \mathbf{x}_{t+1}$ is a chain whose middle node is unobserved. This is why the inference network needs $\mathbf{x}_{t:T}$.
- In the AR DKF, the added edge $\mathbf{x}_{t-1} \rightarrow \mathbf{z}_t$ is a path with no intermediate node, which nothing can block. This is why the AR DKF posterior depends on past observations (Proposition 2).

BIBLIOGRAPHY

[1] Rahul G. Krishnan, Uri Shalit, and David Sontag. Deep Kalman filters. *arXiv preprint arXiv:1511.05121*, 2015. URL https://arxiv.org/abs/1511.05121.

[2] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic Robotics*. MIT Press, Cambridge, MA, 2005.

[3] Kevin P. Murphy. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, Cambridge, MA, 2023.

[4] Rahul G. Krishnan, Uri Shalit, and David Sontag. Structured inference networks for nonlinear state space models. In *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, 2017. URL https://arxiv.org/abs/1609.09869.

[5] Yuri Burda, Roger Grosse, and Ruslan Salakhutdinov. Importance weighted autoencoders. In *International Conference on Learning Representations (ICLR)*, 2016. URL https://arxiv.org/abs/1509.00519.

[6] Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, Cambridge, MA, 2009.